From Imitation to Reasoning to Agentic – Scaling Large Language Models through Reinforcement Learning

Piyush Sawarkar

Founding Member & Tech Lead Text Foundation Models @ BharatGen, India's Sovereign AI National Mission 🇮🇳
IIT Bombay Alumnus

Talk Overview

- The LLM Foundations
- Off-Policy vs On-Policy Learning
- The RL Triad: Algorithms, Environments, and Training Infrastructure
  - The Mathematical RL Formulation and Algorithm Zoo
  - Provenance Mapping key to unlocking and sustaining "RL climb"
  - RL Training Infrastructural Resiliency
  - Verifiable & Unverifiable RL Environments
- Fostering Agentic Behaviours via RL
- Multi-Teacher On Policy Distillation
- Additional Open Problems through Practitioner's Lens
- Case Study on Test Time Compute Efficiency
- RL for Safety, Multilingual & Multicultural Environments
- Case Study on recent OpenAI HF Incident

Talk Overview

The LLM Foundations

Off-Policy vs On-Policy Learning

The RL Triad: Algorithms, Environments, and Training Infrastructure

- The Mathematical RL Formulation and Algorithm Zoo
- Provenance Mapping key to unlocking and sustaining "RL climb"
- RL Training Infrastructural Resiliency
- Verifiable & Unverifiable RL Environments

Fostering Agentic Behaviours via RL

Multi-Teacher On Policy Distillation

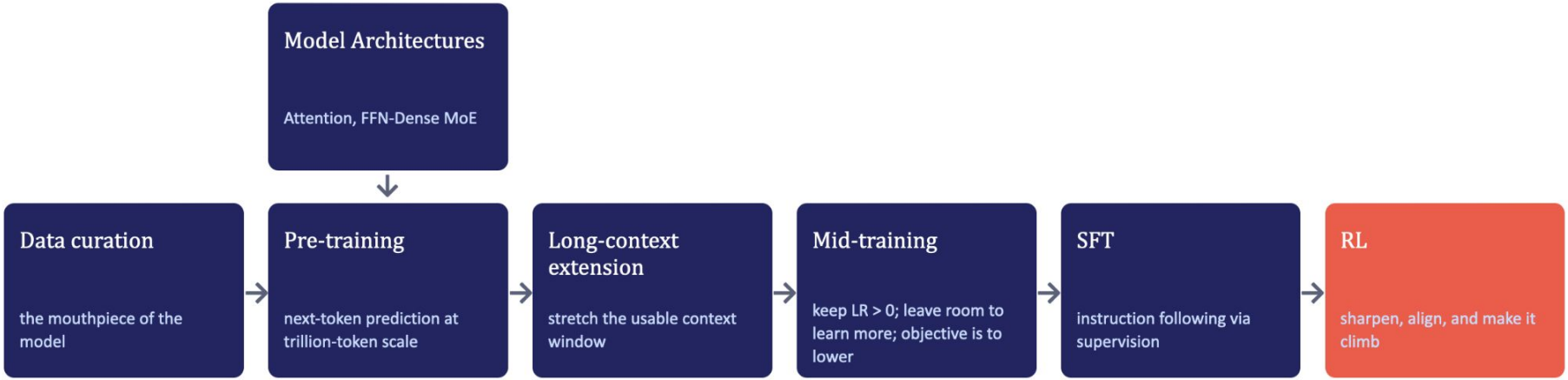
Additional Open Problems through Practitioner's Lens

Case Study on Test Time Compute Efficiency

RL for Safety, Multilingual & Multicultural Environments

Case Study on recent OpenAI HF Incident

The LLM development lifecycle



Two things people underestimate

- Mid-training is a recent, load-bearing phase: a pre-training-style run between pre-training and post-training. Labs deliberately do not decay the learning rate to zero — that headroom is what makes mid-training, sometimes several rounds of it) possible.
- The pipeline is not one-way. If RL stalls, you go back — to SFT, or all the way to mid-training — and climb again.

STAGE 1

Pre-training: knowledge without behaviour

The only objective

$$L = -\Sigma_t \log p(x_t \mid x_{<t})$$

- Cross-entropy at every token: given a context, predict what comes next.
- Trained on trillions of tokens — the knowledge is unquestionably in there.
- But it lives spread across a probability distribution in hyper-dimensional space, not as callable behaviour.

✓ What you get

- World knowledge, language, code, facts
- Raw completion ability: continue any context plausibly

✗ What you don't get

- Instruction adherence — it completes, it does not obey
- Comprehension of a task, planning, building a blueprint
- All the "fancy stuff" you associate with ChatGPT / Claude

STAGE 2

SFT: same loss, two crucial changes

1 The data changes

(instruction, response) pairs in a chat template — the model learns the format of being asked and answering.

2 The loss is masked

Cross-entropy is computed on assistant tokens only. Prompt tokens are masked: learn to generate the whole answer given the context, not to predict the prompt.

One training example, token by token



grey = masked (no loss) **orange = loss computed**

Still token-level cross-entropy, summed over the response — the objective never changed, only what it is pointed at.

THE CATCH

After SFT, your model is still a coin toss

Ask it its own training question, 10 times

Sometimes it answers, sometimes it doesn't. The distribution is still loose — supervision showed it one path, it did not make that path reliable.

"Off-Policy" Distillation teaches confidence, not competence

Train on synthetic answers from DeepSeek or Claude, and the student imitates the teacher's tone of certainty — with or without the underlying ability.

Confidently "solves"

A fourth-order Schrödinger equation

...because that exact solution was in the SFT data.

Then fumbles

Pythagoras with sides 3 and 4

...because imitation does not generalise out of distribution.

Off-Policy vs On-Policy Learning

THE CORE DISTINCTION

Off-policy imitates. On-policy experiences.

OFF-POLICY · pre-training, SFT, off-policy distillation

Learn from someone else's trajectories

- You force another distribution onto the model; the obedient student imitates it.
- Great for breadth: new formats, new domains, new skills.
- Fragile out of distribution — the Pythagoras failure lives here.

ON-POLICY · reinforcement learning

Learn from your own rollouts

- The model generates, gets rewarded, and updates on its own behaviour.
- Don't hand it the solved equation — let it think harder from knowledge it already has from pre-training.
- Like solving problems at home without peeking at the back pages of the book.

Mnemonic: SFT expands what the model can express. RL sharpens what it can reliably do.

Talk Overview

The LLM Foundations

Off-Policy vs On-Policy Learning

The RL Triad: Algorithms, Environments, and Training Infrastructure

- The Mathematical RL Formulation and Algorithm Zoo
- Provenance Mapping key to unlocking and sustaining "RL climb"
- RL Training Infrastructural Resiliency
- Verifiable & Unverifiable RL Environments

Fostering Agentic Behaviours via RL

Multi-Teacher On Policy Distillation

Additional Open Problems through Practitioner's Lens

Case Study on Test Time Compute Efficiency

RL for Safety, Multilingual & Multicultural Environments

Case Study on recent OpenAI HF Incident

Off-Policy vs On-Policy Learning

BEFORE RL CAN WORK

RL needs in-context learning — and ICL comes from scale

ICL is emergent

In-context learning builds up over training; empirically it strengthens with scale. Without it, on-policy exploration has nothing to steer with.

Two ways to scale

More parameters (a bigger brain) — or more FLOPs on the same brain. Both grow capability; only one grows your serving bill forever.

Overfitting? Please.

Success stories of SLMs over Trillions of Tokens of Training exists — performs good on everyday, and agentic tasks. If you manage to overfit an LLM, that itself is an achievement!

The efficiency thesis: you don't need an Einstein's brain if you learn efficiently. Parameter scaling helps — efficient learning substitutes for it.

SFT Expands, RL Squeezes!

RL sharpens a skill the model can already do sometimes. It does not put new knowledge into the weights. For that you need supervised signal from the environment itself.

- RL SQUEEZES, SFT EXPANDS: A COMPARATIVE STUDY OF REASONING LLMS: https://arxiv.org/pdf/2509.21128

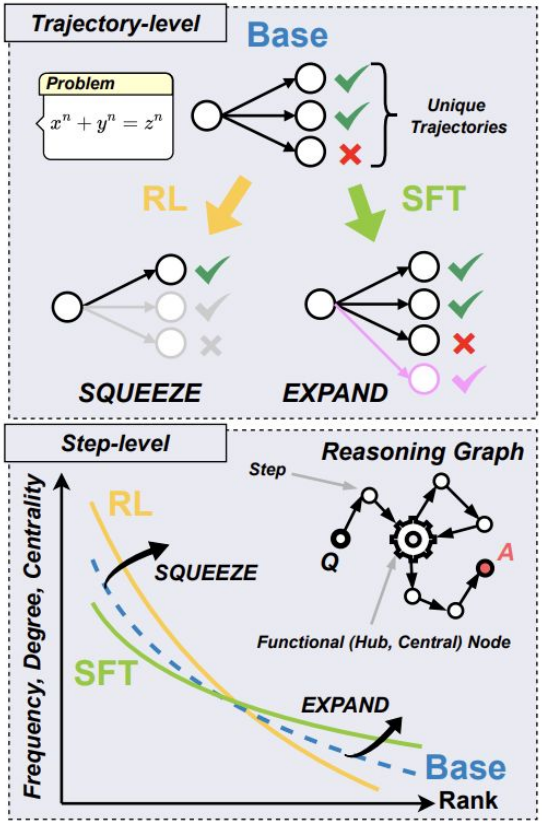


Figure 1: **Overview of our analysis.** (Top) RL compresses incorrect trajectories, and SFT expands correct trajectories. (Bottom) RL concentrates functionality (e.g., hubs) in a small number of steps, and SFT distributes functionality more uniformly across many steps.

RL'S RAZOR: WHY ONLINE REINFORCEMENT LEARNING FORGETS LESS

SFT often achieves new-task gains by erasing prior knowledge, while RL better preserves old skills.

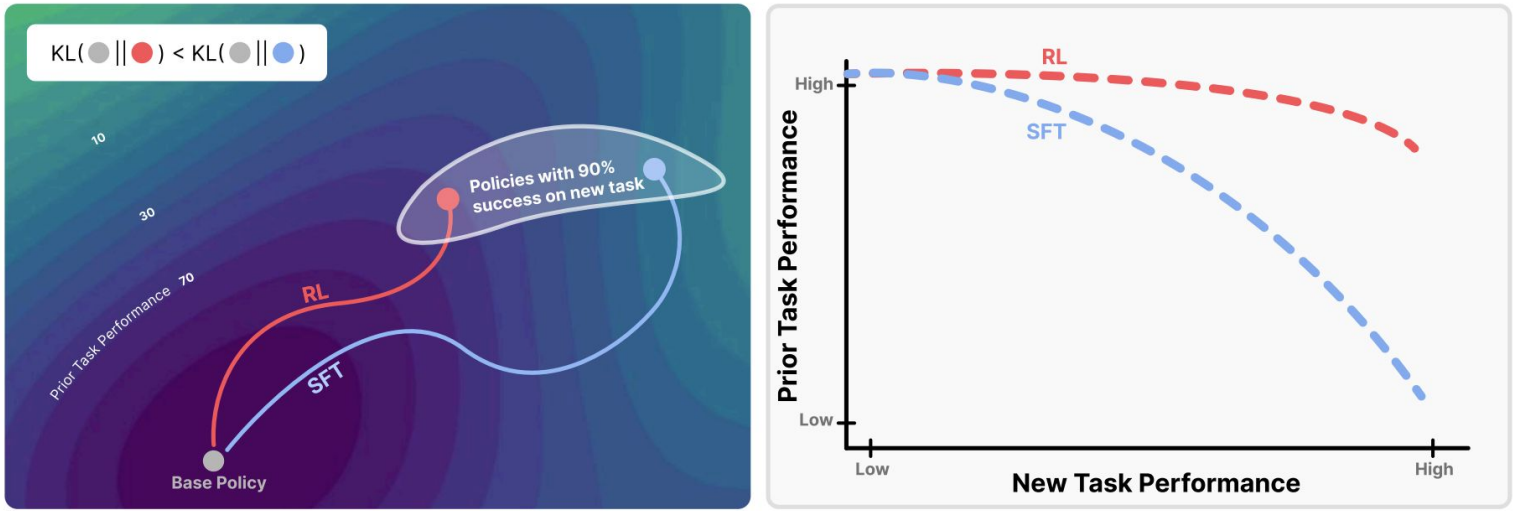
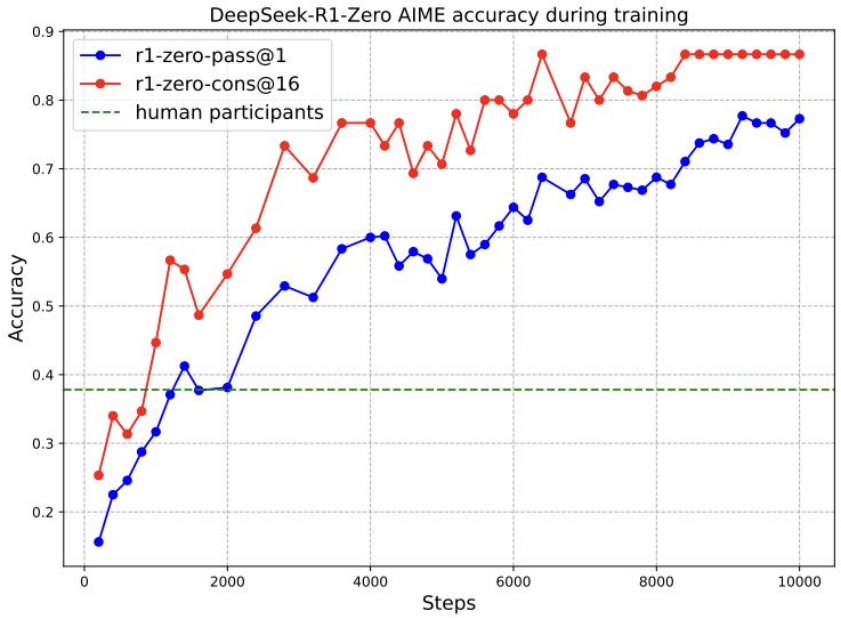


Figure 1: **Bias toward KL-minimal solutions reduces forgetting.** *Left:* Among policies that solve the new task, RL converges to those closest in KL to the base model. *Right:* This KL bias yields higher prior-task retention at matched new-task performance compared to SFT.

https://arxiv.org/abs/2509.04259

DeepSeek R1's attempt to Skip SFT Altogether!



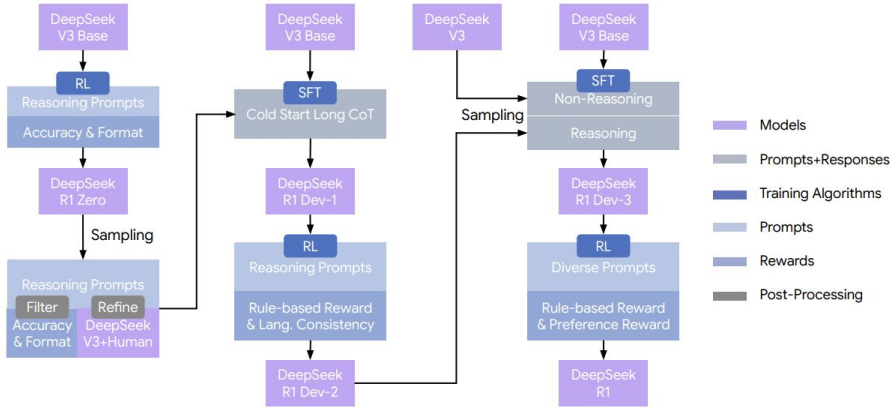


Figure 2 | The multi-stage pipeline of DeepSeek-R1. A detailed background on DeepSeek-V3 Base and DeepSeek-V3 is provided in Supplementary A.1. The models DeepSeek-R1 Dev1, Dev2, and Dev3 represent intermediate checkpoints within this pipeline.

3. DeepSeek-R1

Although DeepSeek-R1-Zero exhibits strong reasoning capabilities, it faces several issues. DeepSeek-R1-Zero struggles with challenges like poor readability, and language mixing, as DeepSeek-V3-Base is trained on multiple languages, especially English and Chinese. To address these issues, we develop DeepSeek-R1, whose pipeline is illustrated in Figure 2.

https://arxiv.org/pdf/2501.12948

Talk Overview

- The LLM Foundations
- Off-Policy vs On-Policy Learning
- The RL Triad: Algorithms, Environments, and Training Infrastructure
  - The Mathematical RL Formulation and Algorithm Zoo
  - Provenance Mapping key to unlocking and sustaining "RL climb"
  - RL Training Infrastructural Resiliency
  - Verifiable & Unverifiable RL Environments
- Fostering Agentic Behaviours via RL
- Multi-Teacher On Policy Distillation
- Additional Open Problems through Practitioner's Lens
- Case Study on Test Time Compute Efficiency
- RL for Safety, Multilingual & Multicultural Environments
- Case Study on recent OpenAI HF Incident

The RL Triad: Algorithms, Environments, and Training Infrastructure

ANATOMY OF AN RL SYSTEM

Three pillars — each one can sink the run

Algorithms

PPO, TRPO, GRPO, DAPO, Dr. GRPO, GSPO, CISPO, OPSD...

Not just a loss function — a full procedure for turning rewards into a safe gradient step. The vanilla objective printed in the paper rarely works on a from-scratch foundation model.

Environments

Verifiable-reward gyms: math, code, tools, agents

Where rollouts happen and rewards are earned. Design them poorly and the model optimises something you did not intend.

Infrastructure

Trainer cluster + rollout cluster + reward plumbing

The RL Training infrastructure considers lot of areas in efficiency, sync vs async, dis-aggregated forward/backward passes

The RL Triad: Algorithms, Environments, and Training Infrastructure

PILLAR 1 · ALGORITHMS

One surrogate objective, an entire family tree

Everything below is the same policy-gradient surrogate with different choices of advantage estimation, ratio clipping, and normalisation.

1

CLASSIC RL

TRPO → PPO

Trust regions, then clipped ratios: take the largest step that is still safe. Needs a learned value/critic model.

→

2

THE DeepSeek MOMENT

GRPO

Drop the critic. Sample a group of rollouts per prompt; the group's mean reward is the baseline. Cheaper, simpler, scales.

→

3

THE REFINEMENTS

DAPO · Dr. GRPO · GSPO · CISPO

Fixes for length bias, clipping asymmetry, token-vs-sequence ratios, and stability at scale.

→

4

ON-POLICY SELF-DISTILLATION

OPSD

Everyone's current favourite: a teacher scores the student's own rollouts — distillation recast as an RL objective.

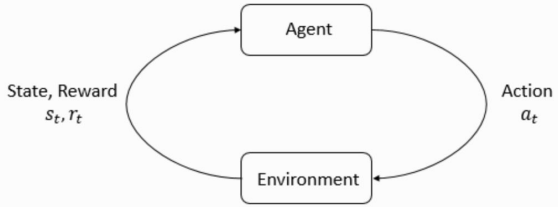
What every variant is really doing: ingest external rewards → estimate an advantage (GAE gives you a family of choices) → take the right gradient step, because only the right step reaches the optimum.

Talk Overview

- The LLM Foundations
- Off-Policy vs On-Policy Learning
- The RL Triad: Algorithms, Environments, and Training Infrastructure
  - The Mathematical RL Formulation and Algorithm Zoo
  - Provenance Mapping key to unlocking and sustaining "RL climb"
  - RL Training Infrastructural Resiliency
  - Verifiable & Unverifiable RL Environments
- Fostering Agentic Behaviours via RL
- Multi-Teacher On Policy Distillation
- Additional Open Problems through Practitioner's Lens
- Case Study on Test Time Compute Efficiency
- RL for Safety, Multilingual & Multicultural Environments
- Case Study on recent OpenAI HF Incident

Setting up the Mathematical Framework for RL

Key Concepts and Terminology



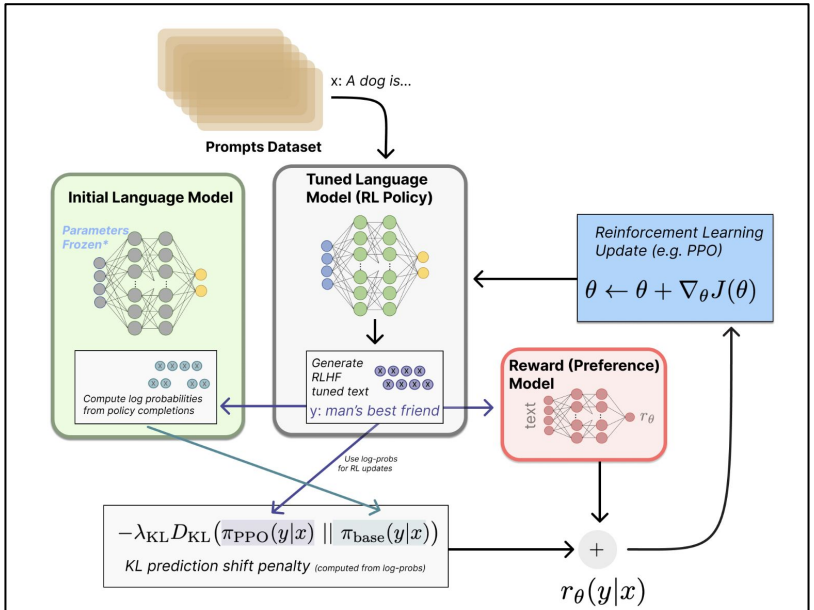
Agent-environment interaction loop.

The main characters of RL are the **agent** and the **environment**. The environment is the world that the agent lives in and interacts with. At every step of interaction, the agent sees a (possibly partial) observation of the state of the world, and then decides on an action to take. The environment changes when the agent acts on it, but may also change on its own.

The agent also perceives a **reward** signal from the environment, a number that tells it how good or bad the current world state is. The goal of the agent is to maximize its cumulative reward, called **return**. Reinforcement learning methods are ways that the agent can learn behaviors to achieve its goal.

To talk more specifically what RL does, we need to introduce additional terminology. We need to talk about

- states and observations,
- action spaces,
- policies,
- trajectories,
- different formulations of return,
- the RL optimization problem,
- and value functions.



The RLHF training pipeline — the RL step optimizes the policy using the reward model signal.

https://spinningup.openai.com/en/latest/spinningup/rl_intro.html
https://rlhfbook.com/c/05-reward-models

Setting up the Mathematical Framework for RL

Markov Decision Processes (MDPs). An MDP is a 5-tuple, $\langle S, A, R, P, \rho_0 \rangle$, where

- S is the set of all valid states,
- A is the set of all valid actions,
- $R : S \times A \times S \to \mathbb{R}$ is the reward function, with $r_t = R(s_t, a_t, s_{t+1})$,
- $P : S \times A \to \mathcal{P}(S)$ is the transition probability function, with $P(s'|s, a)$ being the probability of transitioning into state s' if you start in state s and take action a,
- and ρ_0 is the starting state distribution.

https://spinningup.openai.com/en/latest/spinningup/rl_intro.html

Mathematical Framework for RL

States and Observations

A **state** s is a complete description of the state of the world. There is no information about the world which is hidden from the state. An **observation** o is a partial description of a state, which may omit information.

Action Spaces

Different environments allow different kinds of actions. The set of all valid actions in a given environment is often called the **action space**. Some environments, like Atari and Go, have **discrete action spaces**, where only a finite number of moves are available to the agent. Other environments, like where the agent controls a robot in a physical world, have **continuous action spaces**. In continuous spaces, actions are real-valued vectors.

https://spinningup.openai.com/en/latest/spinningup/rl_intro.html

Mathematical Framework for RL

Policies

A **policy** is a rule used by an agent to decide what actions to take. It can be deterministic, in which case it is usually denoted by μ:

$$a_t = \mu(s_t),$$

or it may be stochastic, in which case it is usually denoted by π:

$$a_t \sim \pi(\cdot|s_t).$$

Because the policy is essentially the agent's brain, it's not uncommon to substitute the word "policy" for "agent", eg saying "The policy is trying to maximize reward."

In deep RL, we deal with **parameterized policies**: policies whose outputs are computable functions that depend on a set of parameters (eg the weights and biases of a neural network) which we can adjust to change the behavior via some optimization algorithm.

We often denote the parameters of such a policy by θ or ϕ, and then write this as a subscript on the policy symbol to highlight the connection:

$$a_t = \mu_\theta(s_t)$$
$$a_t \sim \pi_\theta(\cdot|s_t).$$

Trajectories

A trajectory τ is a sequence of states and actions in the world,

$$\tau = (s_0, a_0, s_1, a_1, \ldots).$$

The very first state of the world, s_0, is randomly sampled from the **start-state distribution**, sometimes denoted by ρ_0:

$$s_0 \sim \rho_0(\cdot).$$

State transitions (what happens to the world between the state at time t, s_t, and the state at $t+1$, s_{t+1}), are governed by the natural laws of the environment, and depend on only the most recent action, a_t. They can be either deterministic,

$$s_{t+1} = f(s_t, a_t)$$

or stochastic,

$$s_{t+1} \sim P(\cdot|s_t, a_t).$$

Actions come from an agent according to its policy.

https://spinningup.openai.com/en/latest/spinningup/rl_intro.html

Mathematical Framework for RL

Reward and Return

The reward function R is critically important in reinforcement learning. It depends on the current state of the world, the action just taken, and the next state of the world:

$$r_t = R(s_t, a_t, s_{t+1})$$

although frequently this is simplified to just a dependence on the current state, $r_t = R(s_t)$, or state-action pair $r_t = R(s_t, a_t)$.

The goal of the agent is to maximize some notion of cumulative reward over a trajectory, but this actually can mean a few things. We'll notate all of these cases with $R(\tau)$, and it will either be clear from context which case we mean, or it won't matter (because the same equations will apply to all cases).

One kind of return is the **finite-horizon undiscounted return**, which is just the sum of rewards obtained in a fixed window of steps:

$$R(\tau) = \sum_{t=0}^{T} r_t.$$

Another kind of return is the **infinite-horizon discounted return**, which is the sum of all rewards *ever* obtained by the agent, but discounted by how far off in the future they're obtained. This formulation of reward includes a discount factor $\gamma \in (0,1)$:

$$R(\tau) = \sum_{t=0}^{\infty} \gamma^t r_t.$$

Why would we ever want a discount factor, though? Don't we just want to get *all* rewards? We do, but the discount factor is both intuitively appealing and mathematically convenient. On an intuitive level: cash now is better than cash later. Mathematically: an infinite-horizon sum of rewards may not converge to a finite value, and is hard to deal with in equations. But with a discount factor and under reasonable conditions, the infinite sum converges.

The RL Problem

Whatever the choice of return measure (whether infinite-horizon discounted, or finite-horizon undiscounted), and whatever the choice of policy, the goal in RL is to select a policy which maximizes **expected return** when the agent acts according to it.

To talk about expected return, we first have to talk about probability distributions over trajectories.

Let's suppose that both the environment transitions and the policy are stochastic. In this case, the probability of a T -step trajectory is:

$$P(\tau|\pi) = \rho_0(s_0) \prod_{t=0}^{T-1} P(s_{t+1}|s_t, a_t)\pi(a_t|s_t).$$

The expected return (for whichever measure), denoted by $J(\pi)$, is then:

$$J(\pi) = \int_{\tau} P(\tau|\pi)R(\tau) = \underset{\tau\sim\pi}{\mathrm{E}}\left[R(\tau)\right].$$

The central optimization problem in RL can then be expressed by

$$\pi^* = \arg\max_{\pi} J(\pi),$$

with π^* being the **optimal policy**.

https://spinningup.openai.com/en/latest/spinningup/rl_intro.html

Mathematical Framework for RL

Value Functions

It's often useful to know the **value** of a state, or state-action pair. By value, we mean the expected return if you start in that state or state-action pair, and then act according to a particular policy forever after. **Value functions** are used, one way or another, in almost every RL algorithm.

There are four main functions of note here.

1. The **On-Policy Value Function**, $V^{\pi}(s)$, which gives the expected return if you start in state s and always act according to policy π:
$$V^{\pi}(s) = \underset{\tau \sim \pi}{\mathrm{E}}\left[R(\tau)\,|s_0 = s\right]$$

2. The **On-Policy Action-Value Function**, $Q^{\pi}(s,a)$, which gives the expected return if you start in state s, take an arbitrary action a (which may not have come from the policy), and then forever after act according to policy π:
$$Q^{\pi}(s,a) = \underset{\tau \sim \pi}{\mathrm{E}}\left[R(\tau)\,|s_0 = s, a_0 = a\right]$$

3. The **Optimal Value Function**, $V^{*}(s)$, which gives the expected return if you start in state s and always act according to the *optimal* policy in the environment:
$$V^{*}(s) = \max_{\pi} \underset{\tau \sim \pi}{\mathrm{E}}\left[R(\tau)\,|s_0 = s\right]$$

4. The **Optimal Action-Value Function**, $Q^{*}(s,a)$, which gives the expected return if you start in state s, take an arbitrary action a, and then forever after act according to the *optimal* policy in the environment:
$$Q^{*}(s,a) = \max_{\pi} \underset{\tau \sim \pi}{\mathrm{E}}\left[R(\tau)\,|s_0 = s, a_0 = a\right]$$

There are two key connections between the value function and the action-value function that come up pretty often:

$$V^{\pi}(s) = \underset{a \sim \pi}{\mathrm{E}}\left[Q^{\pi}(s,a)\right],$$

and

$$V^{*}(s) = \max_{a} Q^{*}(s,a).$$

https://spinningup.openai.com/en/latest/spinningup/rl_intro.html

What are we optimizing?

Choose policy parameters θ that maximize reward **on average** under the current policy.

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)]$$

In practice, we estimate this expectation with sampled rollouts:

$$J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} R(\tau_i), \qquad \tau_i \sim \pi_\theta$$

We craft a gradient/derivative that lets us optimize this.

https://rlhfbook.com/teach/course/lec3-chap6-p1/

Policy Gradient Estimator, Schulman e.t al. 2015

The policy gradient equation

A preview of where we end up:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot \Psi_t\right]$$

The core idea is that we *sample over trials in the environment* and **estimate** the gradient.

Where Ψ_t is the **learning signal** telling the optimizer how good the action was. The choice of Ψ_t determines the algorithm's variance, bias, and compute cost.

How a policy gradient works

Make actions more likely when they lead to better outcomes.

$$\Delta\theta \propto \Psi_t \, \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

Read it as two questions answered at once:

- $\nabla_\theta \log \pi_\theta(a_t \mid s_t)$ — **which action?**
- Ψ_t — **how good was it?** A scalar scoring the outcome.

https://arxiv.org/abs/1506.02438

https://rlhfbook.com/teach/course/lec3-chap6-p1/

Key RL quantities

Three quantities appear throughout this lecture:

Value function $V(s)$: expected future return from state s

$$V(s) = \mathbb{E}\big[G_t \mid S_t = s\big] \quad \text{where } G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

G: Discounted sum of rewards

Action-value $Q(s, a)$: expected return after taking action a in state s

$$Q(s, a) = \mathbb{E}[G_t \mid S_t = s, A_t = a]$$

Advantage $A(s, a) = Q(s, a) - V(s)$: how much better is action a compared to average? Positive → reinforce, negative → suppress, zero → no update.

In RLHF: often $\gamma = 1$ (no discounting) because the unit of optimization is the full completion.

https://rlhfbook.com/teach/course/lec3-chap6-p1/

What Ψ_t can be

Popular choices for Ψ_t (rewards can also be discounted by γ):

	Ψ_t	Description	Variance	Bias
1.	$R(\tau) = \sum_{t=0}^{T} r_t$	Total trajectory reward	Highest	None
2.	$\sum_{t'=t}^{T} r_{t'}$	Future return from t (the return, G_t)	High	None
3.	$G_t - b(s_t)$	Baselined return	Lower	None
4.	$Q^{\pi}(s_t, a_t)$	State-action value function	Med	Depends
5.	$A^{\pi}(s_t, a_t) = Q - V$	Advantage function	Low (with good V)	None
6.	$r_t + \gamma V(s_{t+1}) - V(s_t)$	TD residual	Low	Some

A *baseline* $b(s_t)$ is any value subtracted from the reward signal to reduce variance. We'll show below that this does not change the expected gradient.

https://arxiv.org/abs/1506.02438

https://rlhfbook.com/teach/course/lec3-chap6-p1/

The policy gradient equation

A preview of where we end up:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot \Psi_t\right]$$

The core idea is that we *sample over trials in the environment* and **estimate** the gradient.

Where Ψ_t is the **learning signal** telling the optimizer how good the action was. The choice of Ψ_t determines the algorithm's variance, bias, and compute cost.

Baselines keep the estimator unbiased

Raw returns are noisy. The same action can appear in trajectories with very different total rewards because of later sampled actions and, in classical RL, environment randomness.

The role of a baseline is to **center** that noisy signal: instead of asking "was the return high?", we ask "was it higher or lower than expected for this state?"

Use a centered return:

$$\Psi_t = G_t - b(s_t)$$

where the baseline $b(s_t)$ depends only on the state, not on which action was sampled.

Subtracting this baseline doesn't change the expected gradient:

https://rlhfbook.com/teach/course/lec3-chap6-p1/

The policy gradient estimator

Combining the log-derivative trick, return-to-go, and baseline subtraction:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot \Psi_t\right]$$

This is the **policy gradient theorem**. Every algorithm in this lecture is an instantiation with a specific choice of Ψ_t and regularization.

For language models, this means the sum runs over generated tokens in the completion, while Ψ_t determines how each token is weighted.

Setup: Differentiating an expectation

Ideally, we want $\nabla_\theta J(\theta)$, but we can't do that (the state sampling distribution itself depends on θ).

Written as an integral, the objective is:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)] = \int_\tau p_\theta(\tau) R(\tau)\, d\tau$$

Taking the gradient directly:

$$\nabla_\theta J(\theta) = \int_\tau \nabla_\theta p_\theta(\tau) R(\tau)\, d\tau$$

We can sample trajectories from $p_\theta(\tau)$, but not from $\nabla_\theta p_\theta(\tau)$, since it is not a probability distribution.

The derivation is a few key tricks to let us approximate or compute this.

https://rlhfbook.com/teach/course/lec3-chap6-p1/

Applying the idea of computing the gradient from trajectories

Substituting the log-derivative identity (more on this soon) into the gradient:

$$\begin{aligned}
\nabla_\theta J(\theta) &= \int_\tau \nabla_\theta p_\theta(\tau) R(\tau)\, d\tau && \text{differentiate the objective} \\
&= \int_\tau p_\theta(\tau) \nabla_\theta \log p_\theta(\tau) R(\tau)\, d\tau && \text{log-derivative identity (chain rule)} \\
&= \mathbb{E}_{\tau \sim \pi_\theta}[\nabla_\theta \log p_\theta(\tau) \cdot R(\tau)] && \text{expectation form}
\end{aligned}$$

Now we can estimate this with Monte Carlo sampling!

Applying the idea of computing the gradient from trajectories

The only non-obvious step is the middle line:

$$\begin{aligned}
\nabla_\theta \log p_\theta(\tau) &= \frac{\nabla_\theta p_\theta(\tau)}{p_\theta(\tau)} \\
\Rightarrow \quad \nabla_\theta p_\theta(\tau) &= p_\theta(\tau) \nabla_\theta \log p_\theta(\tau)
\end{aligned}$$

This is the **log-derivative identity**, derived from the chain rule for $\log$.

It rewrites the gradient of a probability in terms of a log-probability, which is much easier to work with.

Applying the idea of computing the gradient from trajectories

Where did the $p_\theta(\tau)$ term go?

It became the **sampling distribution** inside the expectation:

$$\int p_\theta(\tau) f(\tau)\, d\tau = \mathbb{E}_{\tau \sim p_\theta}[f(\tau)]$$

Monte Carlo then estimates that expectation by sampling trajectories from the policy:

$$\mathbb{E}_{\tau \sim p_\theta}[f(\tau)] \approx \frac{1}{N} \sum_{i=1}^{N} f(\tau_i), \qquad \tau_i \sim p_\theta$$

So there is no explicit $p_\theta(\tau_i)$ term in code: **more likely trajectories already appear more often in the sampled batch**.

At a high level, rollout generation handles the sampling, and the loss code handles $R(\tau_i)\, \nabla_\theta \log p_\theta(\tau_i)$.

Expanding log probability of trajectory to compute ∇_θ

The trajectory probability factorizes:

$$p_\theta(\tau) = p(s_0) \prod_{t=0}^{T} \pi_\theta(a_t \mid s_t)\, p(s_{t+1} \mid s_t, a_t)$$

Taking the log:

$$\log p_\theta(\tau) = \log p(s_0) + \sum_{t=0}^{T} \log \pi_\theta(a_t \mid s_t) + \sum_{t=0}^{T} \log p(s_{t+1} \mid s_t, a_t)$$

For language models, this is the familiar autoregressive pattern: a sequence log-probability becomes a sum of token log-probabilities.

https://rlhfbook.com/teach/course/lec3-chap6-p1/

Expanding log probability of trajectory to compute ∇_θ

$$\log p_\theta(\tau) = \log p(s_0) + \sum_{t=0}^{T} \log \pi_\theta(a_t \mid s_t) + \sum_{t=0}^{T} \log p(s_{t+1} \mid s_t, a_t)$$

Now take the gradient w.r.t. θ:

- $\nabla_\theta \log p(s_0) = 0$ — initial state doesn't depend on θ
- $\nabla_\theta \log p(s_{t+1} \mid s_t, a_t) = 0$ — environment dynamics don't depend on θ
- Only $\nabla_\theta \log \pi_\theta(a_t \mid s_t)$ survives!

$$\nabla_\theta \log p_\theta(\tau) = \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

https://rlhfbook.com/teach/course/lec3-chap6-p1/

Mathematical Framework for RL

This was the Policy Gradient derived…

$$\begin{aligned}
\nabla_\theta J(\theta) &= \int_\tau \nabla_\theta p_\theta(\tau) R(\tau)\, d\tau && \text{differentiate the objective} \\
&= \int_\tau p_\theta(\tau) \nabla_\theta \log p_\theta(\tau) R(\tau)\, d\tau && \text{log-derivative identity (chain rule)} \\
&= \mathbb{E}_{\tau \sim \pi_\theta}[\nabla_\theta \log p_\theta(\tau) \cdot R(\tau)] && \text{expectation form}
\end{aligned}$$

From full return to future rewards

An action at time t can't affect past rewards. So instead of weighting by the full trajectory reward $R(\tau)$:

$$\nabla_\theta J = \mathbb{E}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot R(\tau)\right]$$

We can replace $R(\tau)$ with the **return-to-go** $G_t = \sum_{t'=t}^{T} R_{t'}$ — only future rewards from t onward:

$$\nabla_\theta J = \mathbb{E}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot G_t\right]$$

https://rlhfbook.com/teach/course/lec3-chap6-p1/

Final Formulation as summarised in Schulman e.t al. 2015

The policy gradient equation

A preview of where we end up:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot \Psi_t\right]$$

The core idea is that we *sample over trials in the environment* and **estimate** the gradient.

Where Ψ_t is the **learning signal** telling the optimizer how good the action was. The choice of Ψ_t determines the algorithm's variance, bias, and compute cost.

How a policy gradient works

Make actions more likely when they lead to better outcomes.

$$\Delta\theta \propto \Psi_t \, \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

Read it as two questions answered at once:

- $\nabla_\theta \log \pi_\theta(a_t \mid s_t)$ — **which action?**
- Ψ_t — **how good was it?** A scalar scoring the outcome.

https://rlhfbook.com/teach/course/lec3-chap6-p1/

What Ψ_t can be

Popular choices for Ψ_t (rewards can also be discounted by γ):

	Ψ_t	Description	Variance	Bias
1.	$R(\tau) = \sum_{t=0}^{T} r_t$	Total trajectory reward	Highest	None
2.	$\sum_{t'=t}^{T} r_{t'}$	Future return from t (the return, G_t)	High	None
3.	$G_t - b(s_t)$	Baselined return	Lower	None
4.	$Q^{\pi}(s_t, a_t)$	State-action value function	Med	Depends
5.	$A^{\pi}(s_t, a_t) = Q - V$	Advantage function	Low (with good V)	None
6.	$r_t + \gamma V(s_{t+1}) - V(s_t)$	TD residual	Low	Some

A *baseline* $b(s_t)$ is any value subtracted from the reward signal to reduce variance. We'll show below that this does not change the expected gradient.

https://rlhfbook.com/teach/course/lec3-chap6-p1/

The policy gradient equation

A preview of where we end up:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot \Psi_t\right]$$

The core idea is that we *sample over trials in the environment* and **estimate** the gradient.

Where Ψ_t is the **learning signal** telling the optimizer how good the action was. The choice of Ψ_t determines the algorithm's variance, bias, and compute cost.

Baselines keep the estimator unbiased

Raw returns are noisy. The same action can appear in trajectories with very different total rewards because of later sampled actions and, in classical RL, environment randomness.

The role of a baseline is to **center** that noisy signal: instead of asking "was the return high?", we ask "was it higher or lower than expected for this state?"

Use a centered return:

$$\Psi_t = G_t - b(s_t)$$

where the baseline $b(s_t)$ depends only on the state, not on which action was sampled.

Subtracting this baseline doesn't change the expected gradient:

https://rlhfbook.com/teach/course/lec3-chap6-p1/

Baselines keep the estimator unbiased

Raw returns are noisy. The same action can appear in trajectories with very different total rewards because of later sampled actions and, in classical RL, environment randomness.

Use a centered return, $\Psi_t = G_t - b(s_t)$.

Subtracting this baseline doesn't change the expected gradient:

$$\mathbb{E}[\nabla_\theta \log \pi_\theta(a \mid s) \cdot (G_t - b(s))] = \mathbb{E}[\nabla_\theta \log \pi_\theta(a \mid s) \cdot G_t] - \mathbb{E}[\nabla_\theta \log \pi_\theta(a \mid s) \cdot b(s)]$$

The first term is the original estimator. The second vanishes:

$$\begin{aligned}
\mathbb{E}[\nabla_\theta \log \pi_\theta(a \mid s) \cdot b(s)] &= b(s) \cdot \sum_a \nabla_\theta \pi_\theta(a \mid s) \\
&= b(s) \cdot \nabla_\theta \underbrace{\sum_a \pi_\theta(a \mid s)}_{=1} \\
&= 0
\end{aligned}$$

The gradient of a normalized distribution sums to zero — so we're free to subtract **any function of state** as a baseline. This is why Ψ_t options 3–5 reduce variance without introducing bias.

https://rlhfbook.com/teach/course/lec3-chap6-p1/

The policy gradient estimator

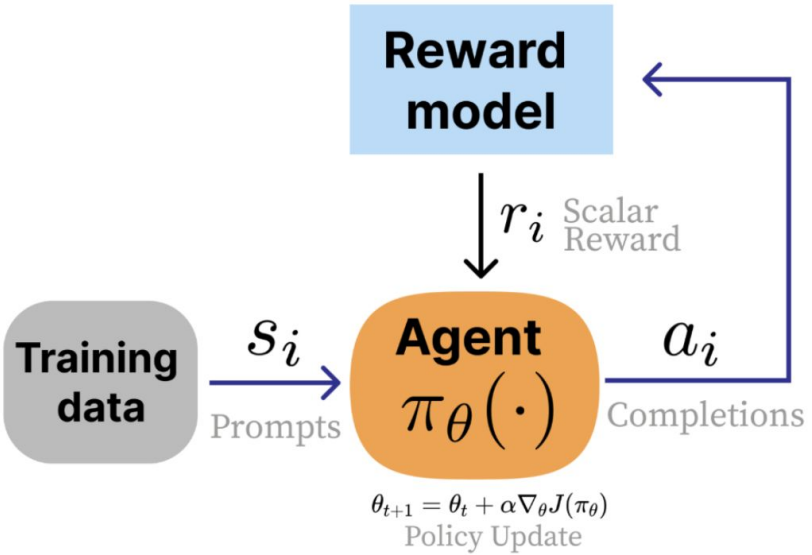
Combining the log-derivative trick, return-to-go, and baseline subtraction:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot \Psi_t\right]$$

This is the **policy gradient theorem**. Every algorithm in this lecture is an instantiation with a specific choice of Ψ_t and regularization.

For language models, this means the sum runs over generated tokens in the completion, while Ψ_t determines how each token is weighted.

RECAP! ...phew!



Policy Gradient Estimator

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot \Psi_t \right]$$

- $\nabla_\theta \log \pi_\theta(a_t \mid s_t)$ — **which action?**
- Ψ_t — **how good was it?** A scalar scoring the outcome.

https://rlhfbook.com/teach/course/lec3-chap6-p1/

Standard Policy Gradient Form

2.1 Policy Gradient Methods

Policy gradient methods work by computing an estimator of the policy gradient and plugging it into a stochastic gradient ascent algorithm. The most commonly used gradient estimator has the form

$$\hat{g} = \hat{\mathbb{E}}_t\Big[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\hat{A}_t\Big] \tag{1}$$

where π_θ is a stochastic policy and $\hat{A}_t$ is an estimator of the advantage function at timestep t. Here, the expectation $\hat{\mathbb{E}}_t[\ldots]$ indicates the empirical average over a finite batch of samples, in an algorithm that alternates between sampling and optimization. Implementations that use automatic differentiation software work by constructing an objective function whose gradient is the policy gradient estimator; the estimator $\hat{g}$ is obtained by differentiating the objective

$$L^{PG}(\theta) = \hat{\mathbb{E}}_t\Big[\log \pi_\theta(a_t \mid s_t)\hat{A}_t\Big]. \tag{2}$$

While it is appealing to perform multiple steps of optimization on this loss L^{PG} using the same trajectory, doing so is not well-justified, and empirically it often leads to destructively large policy updates (see Section 6.1; results are not shown but were similar or worse than the "no clipping or penalty" setting).

https://arxiv.org/pdf/1707.06347

Surrogate as derived in TRPO by Schulman et. al.

2.2 Trust Region Methods

In TRPO [Sch+15b], an objective function (the "surrogate" objective) is maximized subject to a constraint on the size of the policy update. Specifically,

$$\underset{\theta}{\text{maximize}} \quad \hat{\mathbb{E}}_t\left[\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}\hat{A}_t\right] \tag{3}$$

$$\text{subject to} \quad \hat{\mathbb{E}}_t[\mathrm{KL}[\pi_{\theta_{\text{old}}}(\cdot \mid s_t), \pi_\theta(\cdot \mid s_t)]] \leq \delta. \tag{4}$$

Here, θ_{old} is the vector of policy parameters before the update. This problem can efficiently be approximately solved using the conjugate gradient algorithm, after making a linear approximation to the objective and a quadratic approximation to the constraint.

The theory justifying TRPO actually suggests using a penalty instead of a constraint, i.e., solving the unconstrained optimization problem

$$\underset{\theta}{\text{maximize}}\, \hat{\mathbb{E}}_t\left[\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}\hat{A}_t - \beta\, \mathrm{KL}[\pi_{\theta_{\text{old}}}(\cdot \mid s_t), \pi_\theta(\cdot \mid s_t)]\right] \tag{5}$$

for some coefficient β. This follows from the fact that a certain surrogate objective (which computes the max KL over states instead of the mean) forms a lower bound (i.e., a pessimistic bound) on the performance of the policy π. TRPO uses a hard constraint rather than a penalty because it is hard to choose a single value of β that performs well across different problems—or even within a single problem, where the the characteristics change over the course of learning. Hence, to achieve our goal of a first-order algorithm that emulates the monotonic improvement of TRPO, experiments show that it is not sufficient to simply choose a fixed penalty coefficient β and optimize the penalized objective Equation (5) with SGD; additional modifications are required.

https://arxiv.org/pdf/1502.05477

Proximal Policy Optimisation

2.1 Proximal Policy Optimization (PPO)

PPO [21] introduces a clipped surrogate objective for policy optimization. By constraining the policy updates within a proximal region of the previous policy using clip, PPO stabilizes training and improves sample efficiency. Specifically, PPO updates the policy by maximizing the following objective:

$$\mathcal{J}_{\text{PPO}}(\theta) = \mathbb{E}_{(q,a)\sim\mathcal{D},o_{\leq t}\sim\pi_{\theta_{\text{old}}}(\cdot|q)}\left[\min\left(\frac{\pi_\theta(o_t \mid q,o_{<t})}{\pi_{\theta_{\text{old}}}(o_t \mid q,o_{<t})}\hat{A}_t,\ \text{clip}\left(\frac{\pi_\theta(o_t \mid q,o_{<t})}{\pi_{\theta_{\text{old}}}(o_t \mid q,o_{<t})},1-\varepsilon,1+\varepsilon\right)\hat{A}_t\right)\right], \quad (1)$$

where (q,a) is a question-answer pair from the data distribution $\mathcal{D}$, ε is the clipping range of importance sampling ratio, and $\hat{A}_t$ is an estimator of the advantage at time step t. Given the value function V and the reward function R, $\hat{A}_t$ is computed using the Generalized Advantage Estimation (GAE) [22]:

$$\hat{A}_t^{\text{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty}(\gamma\lambda)^l\delta_{t+l}, \quad (2)$$

where

$$\delta_l = R_l + \gamma V(s_{l+1}) - V(s_l), \quad 0 \leq \gamma,\lambda \leq 1. \quad (3)$$

DAPO: https://arxiv.org/pdf/2503.14476v1 PPO: https://arxiv.org/pdf/1707.06347

GRPO objective

Clipped ratio (like PPO) + group-normalized advantages + KL penalty directly in loss:

$$J(\theta) = \frac{1}{G}\sum_{i=1}^{G}\frac{1}{|a_i|}\sum_{t=1}^{|a_i|}\left(\min\left(\rho_{i,t}\hat{A}_i,\ \text{clip}(\rho_{i,t}, 1-\varepsilon, 1+\varepsilon)\hat{A}_i\right) - \beta\, D_{\text{KL},i,t}\right)$$

$$\hat{A}_i = \frac{R_i - \text{mean}(R_1, \ldots, R_G)}{\text{std}(R_1, \ldots, R_G)}$$

Where the clipping applies **per-token**: $\rho_{i,t} = \frac{\pi_\theta(a_{i,t}|s_t)}{\pi_{\theta_{\text{old}}}(a_{i,t}|s_t)}$, but $\hat{A}_i$ is shared across all tokens in the completion (sequence-level advantage, per-token ratio).

https://rlhfbook.com/teach/course/lec3-chap6-p1/, https://arxiv.org/pdf/2402.03300, https://rlhfbook.com/c/06-policy-gradients

Comparison between PPO and GRPO

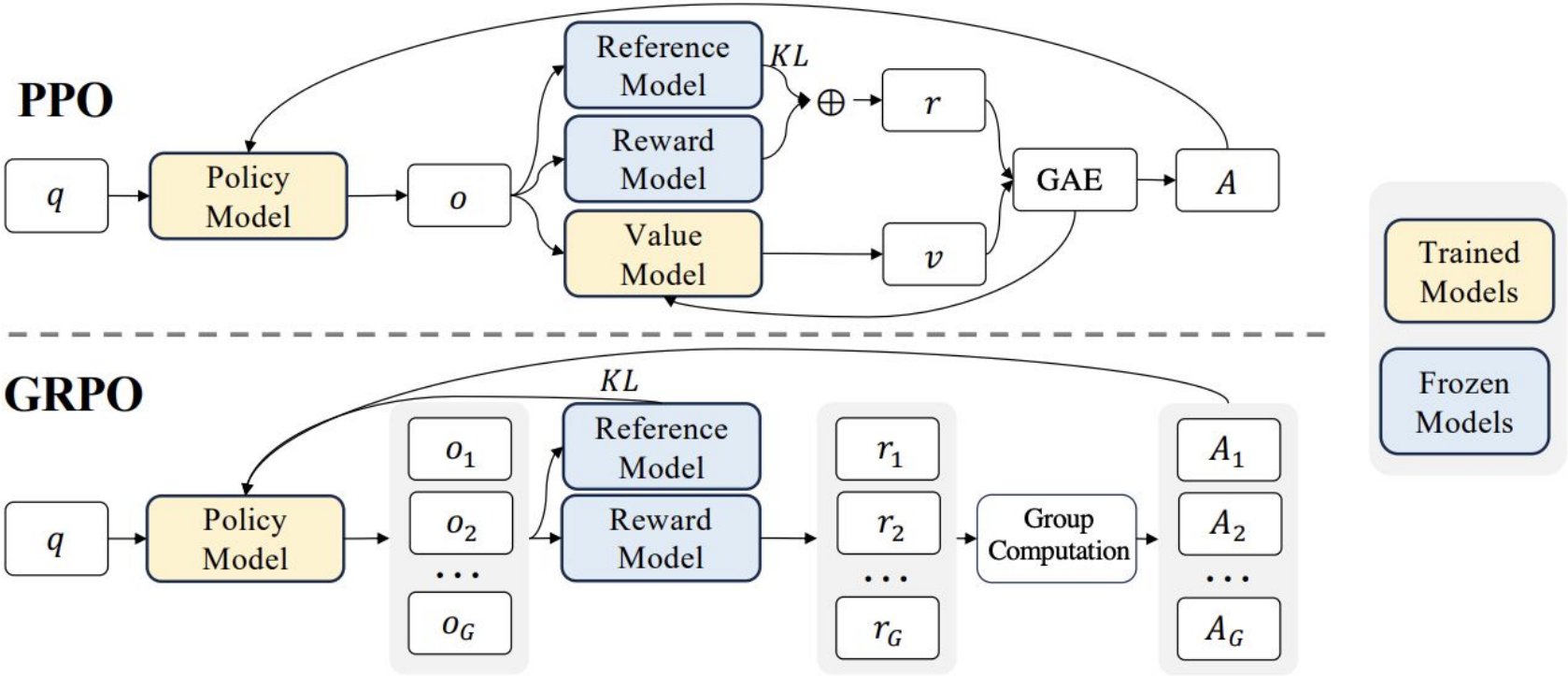


Figure 4 | Demonstration of PPO and our GRPO. GRPO foregoes the value model, instead estimating the baseline from group scores, significantly reducing training resources.

https://arxiv.org/pdf/2402.03300

RL Algorithm Efficiency Wise Comparison

GRPO vs PPO

	PPO	GRPO
Value function	Learned V_ϕ	None
Advantage	Per-token via GAE	Sequence-level, group z-score
KL penalty	In reward (before advantages)	In loss (default, but optional)
Models in memory	4 (policy, value, ref, RM)	3 (policy, ref, RM) or 2 without KL
Complexity	Higher	Lower
Popular for	General RLHF	Reasoning / RLVR (DeepSeek R1)

GRPO is PPO minus the value function, with a statistical baseline instead.

https://rlhfbook.com/teach/course/lec3-chap6-p1/, https://arxiv.org/pdf/2402.03300, https://rlhfbook.com/c/06-policy-gradients

GSPO: Sequence-level ratios

Problem: aggregating per-token importance ratios across long sequences is numerically unstable. A single token with a large ratio can dominate the update.

GSPO uses a geometric mean — a single, length-normalized importance weight per response:

$$\rho_i(\theta) = \exp\left(\frac{1}{|a_i|}\sum_{t=1}^{|a_i|}\log\frac{\pi_\theta(a_{i,t} \mid s)}{\pi_{\theta_{\text{old}}}(a_{i,t} \mid s)}\right)$$

The geometric mean stays in a reasonable numerical range for any sequence length — similar gradient signal, better numerics. The full objective mirrors GRPO but with the sequence-level ratio:

$$J_{\text{GSPO}}(\theta) = \frac{1}{G}\sum_{i=1}^{G}\min\Big(\rho_i(\theta)\hat{A}_i,\ \text{clip}(\rho_i(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_i\Big)$$

The clipping range ε now operates on a per-token average scale, making it comparable across different completion lengths.

https://arxiv.org/pdf/2507.18071, https://rlhfbook.com/teach/course/lec3-chap6-p1/

Zheng et al., 2025

3 DAPO

We propose the **D**ecouple Clip and **D**ynamic s**A**mpling **P**olicy **O**ptimization (DAPO) algorithm. DAPO samples a group of outputs $\{o_i\}_{i=1}^{G}$ for each question q paired with the answer a, and optimizes the policy via the following objective:

$$\begin{aligned} \mathcal{J}_{\text{DAPO}}(\theta) = \; & \mathbb{E}_{(q,a)\sim\mathcal{D},\{o_i\}_{i=1}^{G}\sim\pi_{\theta_{\text{old}}}(\cdot|q)} \\ & \left[\frac{1}{\sum_{i=1}^{G}|o_i|}\sum_{i=1}^{G}\sum_{t=1}^{|o_i|}\min\left(r_{i,t}(\theta)\hat{A}_{i,t},\ \text{clip}\left(r_{i,t}(\theta),1-\varepsilon_{\text{low}},1+\varepsilon_{\text{high}}\right)\hat{A}_{i,t}\right)\right] \\ \text{s.t.} \quad & 0 < \left|\{o_i \mid \texttt{is_equivalent}(a,o_i)\}\right| < G, \end{aligned} \tag{8}$$

where

$$r_{i,t}(\theta) = \frac{\pi_\theta(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})}, \quad \hat{A}_{i,t} = \frac{R_i - \text{mean}(\{R_i\}_{i=1}^{G})}{\text{std}(\{R_i\}_{i=1}^{G})}. \tag{9}$$

The full algorithm can be found in Algorithm 1. In this section, we will introduce the key techniques associated with DAPO.

Dr. GRPO, 2025

3.1 GRPO Leads to Biased Optimization

In Deepseek-R1-Zero (Guo et al., 2025), a notable trend is the consistent increase in response length throughout the training process. This is frequently interpreted as an indication of the development of advanced reasoning abilities such as self-reflection. Recent studies (Pan et al., 2025; Zeng et al., 2025; Hu et al., 2025) have replicated this phenomenon using various algorithms and implementations. However, we argue that **the observed increase in response length may also be attributed to a bias inherent in the GRPO (Shao et al., 2024) objective function**:

$$\mathcal{J}_{GRPO}(\pi_\theta) = \mathbb{E}_{\mathbf{q}\sim p_Q,\{\mathbf{o}_i\}_{i=1}^{G}\sim\pi_{\theta_{old}}(\cdot|\mathbf{q})} \frac{1}{G}\sum_{i=1}^{G}\frac{1}{|\mathbf{o}_i|}\sum_{t=1}^{|\mathbf{o}_i|}\left\{\min\left[\frac{\pi_\theta(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}\hat{A}_{i,t}, \text{clip}\left(\frac{\pi_\theta(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}, 1-\epsilon, 1+\epsilon\right)\hat{A}_{i,t}\right]\right\}, \quad (3)$$

where

$$\hat{A}_{i,t} = \frac{R(\mathbf{q},\mathbf{o}_i) - \text{mean}(\{R(\mathbf{q},\mathbf{o}_1),\ldots,R(\mathbf{q},\mathbf{o}_G)\})}{\text{std}(\{R(\mathbf{q},\mathbf{o}_1),\ldots,R(\mathbf{q},\mathbf{o}_G)\})},$$

with the return $R(\mathbf{q},\mathbf{o}_i)$ typically only including the *outcome verifiable reward* in LLM reasoning (the analysis also applies to process reward cases).

Compared to the objective function in Eq. (2), GRPO introduces two biases (see also Fig. 4):

- **Response-level length bias**: This arises from dividing by $|\mathbf{o}_i|$. For positive advantages ($\hat{A}_{i,t} > 0$, indicating a correct response), this bias results in greater gradient updates for shorter responses, leading the policy to favor brevity in correct answers. Conversely, for negative advantages ($\hat{A}_{i,t} < 0$, indicating an incorrect response), longer responses are penalized less due to their larger $|\mathbf{o}_i|$, causing the policy to prefer lengthier responses among incorrect ones.
- **Question-level difficulty bias**: This is caused by dividing the centered outcome reward by $\text{std}(\{R(\mathbf{q},\mathbf{o}_1),\ldots,R(\mathbf{q},\mathbf{o}_G)\})$. Questions with lower standard deviations (e.g., those that are too easy or too hard, with the outcome rewards being almost all 1 or 0) are given higher weights during policy updates. While advantage normalization is a common trick in RL (Andrychowicz et al., 2021), it is typically computed across an entire batch. In contrast, question-level normalization results in varying weights in the objective for different questions, leading to a difficulty bias in optimization.

3.2 Dr. GRPO: Group Relative Policy Optimization Done Right

To avoid the aforementioned optimization bias in GRPO, we propose to simply remove the $\frac{1}{|\mathbf{o}_i|}$ and $\text{std}(\{R(\mathbf{q},\mathbf{o}_1),\ldots,R(\mathbf{q},\mathbf{o}_G)\})$ normalization terms.

GRPO

$$\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|\mathbf{o}_i|}\sum_{t=1}^{|\mathbf{o}_i|}\left\{\min\left[\frac{\pi_\theta(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}\hat{A}_{i,t}, \text{clip}\left(\frac{\pi_\theta(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}, 1-\epsilon, 1+\epsilon\right)\hat{A}_{i,t}\right]\right\},$$

$$\text{where } \hat{A}_{i,t} = \frac{R(\mathbf{q},\mathbf{o}_i) - \text{mean}(\{R(\mathbf{q},\mathbf{o}_1),\ldots,R(\mathbf{q},\mathbf{o}_G)\})}{\text{std}(\{R(\mathbf{q},\mathbf{o}_1),\ldots,R(\mathbf{q},\mathbf{o}_G)\})}.$$

Dr. GRPO

GRPO Done Right (without bias)

$$\frac{1}{G}\sum_{i=1}^{G}\sum_{t=1}^{|\mathbf{o}_i|}\left\{\min\left[\frac{\pi_\theta(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}\hat{A}_{i,t}, \text{clip}\left(\frac{\pi_\theta(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}, 1-\epsilon, 1+\epsilon\right)\hat{A}_{i,t}\right]\right\},$$

$$\text{where } \hat{A}_{i,t} = R(\mathbf{q},\mathbf{o}_i) - \text{mean}(\{R(\mathbf{q},\mathbf{o}_1),\ldots,R(\mathbf{q},\mathbf{o}_G)\}).$$

https://arxiv.org/pdf/2503.20783

Standard Distillation Problem and its Drawbacks

KD Problem

3 Distillation for Auto-regressive Sequence Models

Problem Setup. We are given two auto-regressive sequence models of different capacity, where p_S and p_T refers to the student and teacher respectively. We assume that the student has learnable parameters θ and p_S^θ is differentiable w.r.t θ. We are also given a dataset of inputs X. Optionally, we can also assume access to a dataset of input-output sequence pairs (X, Y). If not given, such a dataset can be generated by sampling sequences from the teacher. For a divergence $\mathcal{D}$, we define the discrepancy between token-level distributions of p_T and p_S as

$$\mathcal{D}\big(p_T \| p_S^\theta\big)(y|x) := \frac{1}{L_y} \sum_{n=1}^{L_y} \mathcal{D}\big(p_T(\cdot|y_{<n}, x) \| p_S^\theta(\cdot|y_{<n}, x)\big), \tag{2}$$

for an input x and output sequence y. For example, using JSD(β) as $\mathcal{D}$ in equation 2 results in $\mathcal{D}_{JSD(\beta)}\big(p_T \| p_S^\theta\big)(y|x) = \frac{1}{L_y} \sum_n \mathcal{D}_{JSD(\beta)}\big(p_T(\cdot|y_{<n}, x) \big\| p_S^\theta(\cdot|y_{<n}, x)\big)$.

Supervised FT. If we are only given a fixed dataset of ground-truth output sequences but not query access to the teacher policy, then a simple approach is to minimize the negative log-likelihood of such sequences under the student policy: $L_{SFT}(\theta) = \mathbb{E}_{(x,y)\sim(X,Y)}\big[-\log p_S^\theta(y|x)\big]$.

Sequence-Level KD (Kim & Rush, 2016). SeqKD maximizes the likelihood of high probability sequences generated by the teacher, and can be viewed as supervised FT on teacher-generated outputs.

Supervised KD (Hinton et al., 2015; Sanh et al., 2019) is a widely used technique where the student is trained to imitate the token-level probability distributions of the teacher. The student p_S is trained with the supervised objective L_{SD} over the target token-level probabilities of the teacher p_T:

$$L_{SD}(\theta) := \mathbb{E}_{(x,y)\sim(X,Y)}\Big[\mathcal{D}_{KL}\big(p_T \| p_S^\theta\big)(y|x)\Big], \tag{3}$$

where the expectation is over the samples from the dataset. This supervised objective results in a rich training signal by leveraging the full token-level distribution of the teacher.

Drawbacks of being Off-policy

As discussed above, commonly-used KD approaches use a fixed dataset of output sequences, either using ground-truth targets or teacher-generated sequences. However, distilling auto-regressive student models using such approaches results in train-inference distribution mismatch. This is because the partial sequences encountered by the student during the auto-regressive generation phase at inference can be quite different from the ones seen during the training phase. Since predictions at any step are contingent upon previous steps in auto-regressive models, this mismatch can have a cascading effect where error in prediction at early step can affect the future predictions, resulting in poor quality text generation. To address this mismatch, we draw heavily from imitation learning (IL). In particular, on-policy imitation approaches (*e.g.* Ross et al., 2011) iteratively collect sequences using the student policy, obtain expert labels for those sequences, and then retrain the student on this dataset. Despite their popularity in robotics and deep RL (Parisotto et al., 2015; Kelly et al., 2019; Agarwal et al., 2022), on-policy approaches are typically not used for distilling auto-regressive models.

- **On-policy training** samples rollouts from the student model itself, and assigns them some reward.
- **Off-policy training** relies on target outputs from some external source that the student learns to imitate.

https://arxiv.org/pdf/2306.13649, https://thinkingmachines.ai/blog/on-policy-distillation/

Proposition for On-Policy Distillation

Extending on-policy imitation to distillation, we present **on-policy KD**. When using on-policy data during distillation, the student receives token-specific feedback from the teacher's logits on the erroneous tokens in its self-generated output sequences. This enables a form of feedback loop akin to what we observe in RL, which helps minimize the train-inference distribution mismatch. Moreover, as the student evolves during training, the data it generates also improves in quality. Given an input x, the student generates the output sequence y and imitates the teacher token-level distributions, $p_T(y_n|x)$, on intermediate states $y_{<n}$. Specifically, the on-policy loss $\mathcal{L}_{OD}$ is given by

$$L_{OD}(\theta) := \mathbb{E}_{x\sim X}\left[\mathbb{E}_{y\sim p_S(\cdot|x)}\left[\mathcal{D}_{KL}\left(p_T \| p_S^\theta\right)(y|x)\right]\right], \tag{4}$$

You are a better judge of yourself!

KL-Based Divergences. The divergence between two probability distributions is a measure of the similarity of the distributions, with KL divergence a prevalent measure. The KL divergence between two discrete distributions $P(\mathcal{C})$ and $Q(\mathcal{C})$ is given by: $\mathcal{D}_{KL}(P\|Q) = \sum_{c\in\mathcal{C}} P(c)\log\frac{P(c)}{Q(c)}$.

The KL divergence is not symmetric: $\mathcal{D}_{KL}(P\|Q) \neq \mathcal{D}_{KL}(Q\|P)$. As such, we refer to $\mathcal{D}_{KL}(P\|Q)$ as the **forward KL** while $\mathcal{D}_{KL}(Q\|P)$ as the **reverse KL** between P and Q. Forward KL under an empirical data distribution corresponds to maximum likelihood, which we optimize in supervised learning. Given model capacity mismatch, when approximating $P(\mathcal{C})$ using a distribution $Q_\theta(\mathcal{C})$, minimizing the reverse and forward KL results in mean and mode-seeking behavior (Figure A.16).

While KL divergence can be unbounded, a well-known divergence that is *bounded* even for probability distributions with disjoint supports is the **generalized JSD** (Jensen-Shannon divergence). JSD(β) interpolates between the forward and reverse KL using the bounded coefficient $0 < \beta < 1$:

$$\mathcal{D}_{JSD(\beta)}(P\|Q) = \beta\mathcal{D}_{KL}\Big(P\Big\|\beta P + (1-\beta)Q\Big) + (1-\beta)\mathcal{D}_{KL}\Big(Q\Big\|\beta P + (1-\beta)Q\Big) \tag{1}$$

Huszár (2015) show that $\lim_{\beta\to 0}\mathcal{D}_{JSD(\beta)}(P\|Q)/\beta = \mathcal{D}_{KL}(P\|Q)$. As such, gradients of JSD(β) behave similarly to forward KL and reverse KL when β is close to 0 and 1 respectively.

https://arxiv.org/pdf/2306.13649

On Policy Self Distillation Background

On-policy self-distillation and the geometry of KL

OPSD: the teacher grades your homework

- The student generates rollouts on-policy — its own distribution, its own mistakes.
- A teacher model supervises those exact trajectories, token by token, instead of supplying its own.
- Best of both worlds: dense supervision like SFT, on-policy grounding like RL.
- Multi-teacher variants: train RL experts per domain, then distill them into one student — distillation itself modelled as an RL objective.

Reverse KL · $KL(\text{student} \| \text{teacher})$

Mode-seeking. The student commits to one high-probability mode of the teacher. Sharp, decisive — the natural fit for on-policy distillation.

Forward KL · $KL(\text{teacher} \| \text{student})$

Mean-seeking. The student spreads mass over everything the teacher covers. Broad but blurry — closer to classic SFT-style distillation.

You are a better judge of yourself!

https://arxiv.org/pdf/2512.13961, https://arxiv.org/pdf/2306.13649, https://thinkingmachines.ai/blog/on-policy-distillation/, https://arxiv.org/pdf/2606.30406v1, https://thinkingmachines.ai/news/learning-to-replicate-expert-judgment-in-financial-tasks/, https://arxiv.org/abs/2606.19348

KD but with R-KL – Mean Seeking And Mode Seeking Behaviour!

A.7 Mode-seeking vs Mode-covering KL

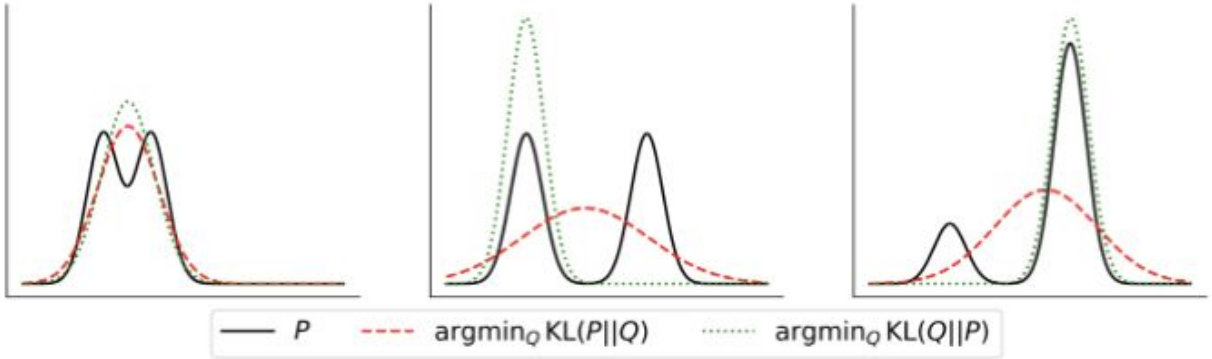


Figure A.16: **Mode-seeking *vs* Model-covering KL with capacity mismatch**. We show the learned distribution Q_θ when minimizing the forward and reverse KL w.r.t Q between a mixture distribution P and a unimodal Gaussian Q_θ. Reverse KL is mode-seeking as it forces Q_θ to be zero where P is zero and hence makes it concentrate on one of the modes (last plot). However, forward KL is mode-covering as it ensures that there is some mass under Q_θ wherever there is some mass under P. See Le (2017) to replicate this plot.

https://arxiv.org/pdf/2306.13649

On Policy Self Distillation (OPSD)

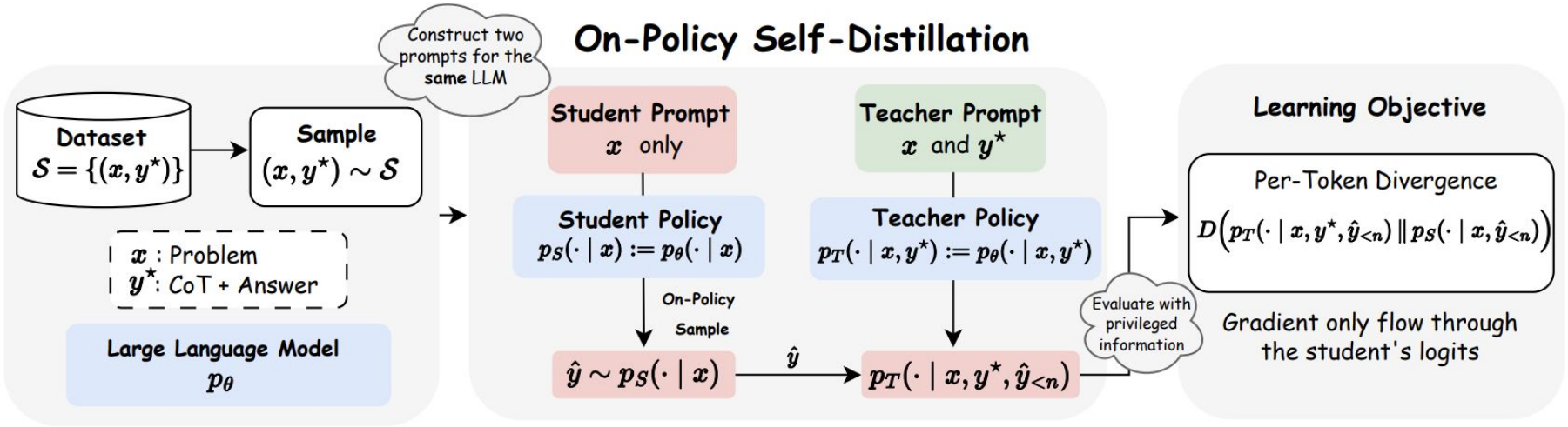


Figure 1. **Overview of On-Policy Self-Distillation (OPSD):** Given a reasoning dataset $\mathcal{S} = \{(x_i, y_i^\star)\}_{i=1}^N$, we instantiate two policies from the same LLM: a *student policy* $p_S(\cdot \mid x)$ and a *teacher policy* $p_T(\cdot \mid x, y^\star)$. The student generates an on-policy response $\hat{y} \sim p_S(\cdot \mid x)$. Both policies then evaluate this trajectory to produce next-token distributions $p_S(\cdot \mid x, \hat{y}_{<n})$ and $p_T(\cdot \mid x, y^\star, \hat{y}_{<n})$ at each step n. The learning objective minimizes the per-token divergence $D(p_T \| p_S)$ along the student's rollout. The divergence here can be forward KL, reverse KL or JSD. Crucially, gradients backpropagate only through the student's logits, allowing the model to self-distil.

Has a hyperparameter that the Frozen Teacher needs to be updated!

https://arxiv.org/pdf/2601.18734

Self-Distillation Policy Optimisation

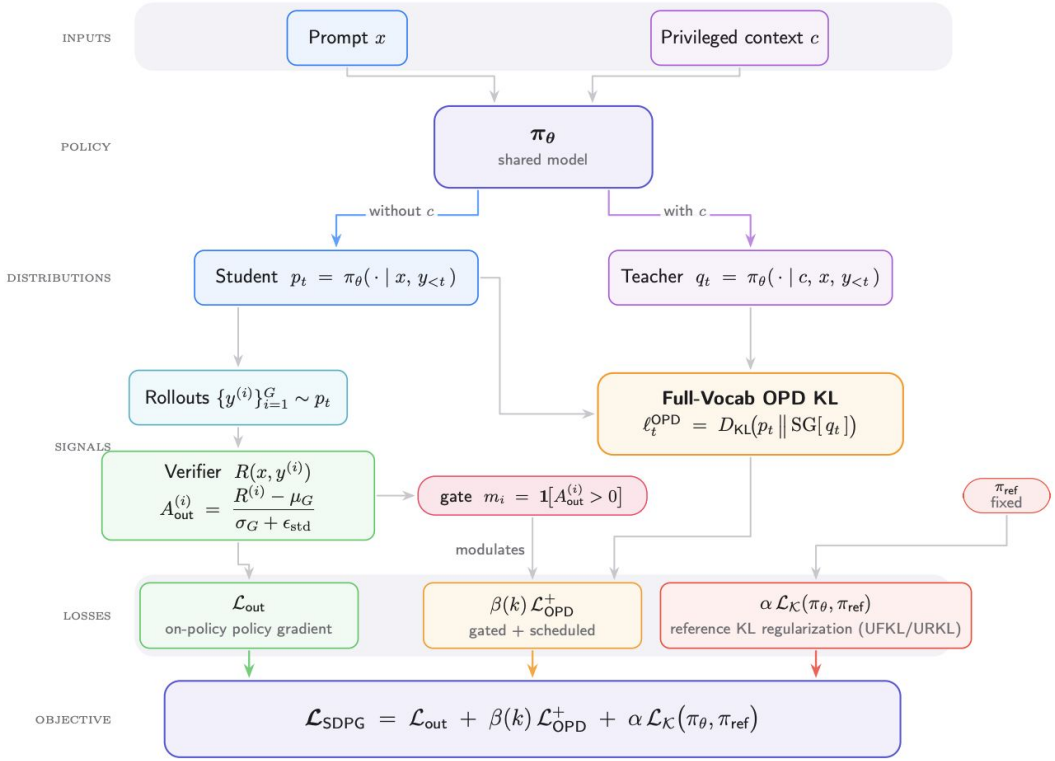
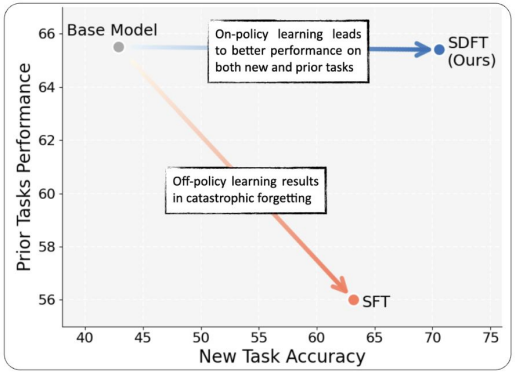
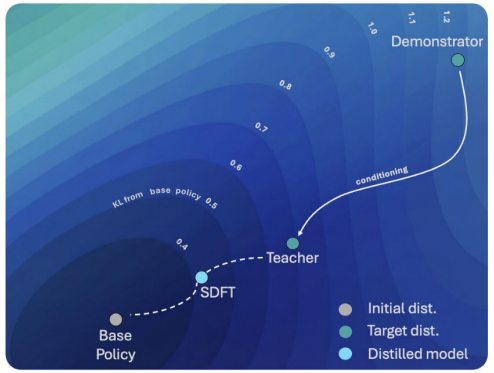


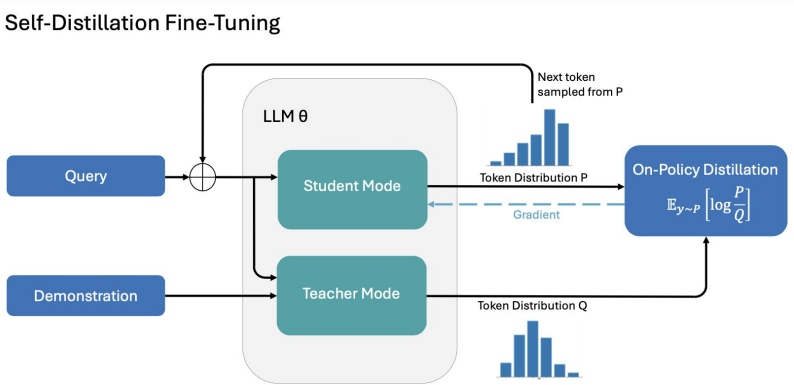
Figure 1: Overview of the Self-Distilled Policy Gradient (SDPG) objective, combining rollout-based outcome optimization, gated On-Policy Distillation (OPD) from privileged context, and a KL regularization to a fixed reference policy. Note that OPD is also a form of policy gradient.

https://arxiv.org/pdf/2606.04036

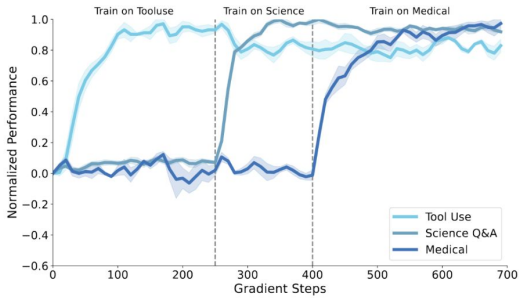
Self-Distillation Enables Continual Learning



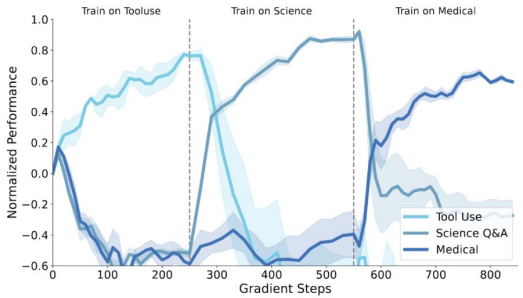




Learning without forgetting



(a) SDFT



(b) SFT

https://self-distillation.github.io/SDFT

In a challenging continual learning experiment, we train a **single model sequentially** on **three different tasks (Tool Use → Science → Medical)**.

SDFT successfully learns each new task while retaining performance on previous ones. In contrast, SFT exhibits severe interference—performance on earlier skills rapidly degrades once training shifts to a new task. This demonstrates that SDFT supports true continual learning, allowing incremental skill acquisition without catastrophic forgetting.

Case Study: OpenClaw-RL
#Recursive Self Improvement #Continual Learning #LLM-Personalisation

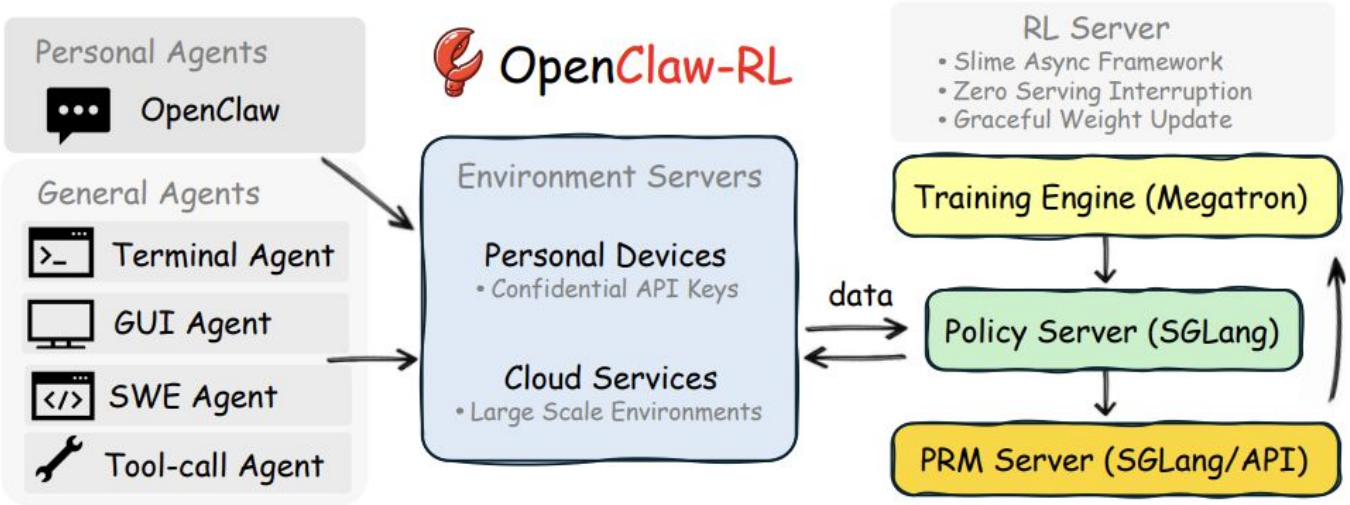


Figure 1 | **OpenClaw-RL infrastructure overview.** Interaction streams come from two agent types: Personal Agents (conversational, personalized), hosted on personal devices, and General Agents (terminal, GUI, SWE, and tool-call agents), hosted on cloud services. The collected samples flow into our RL server built on the asynchronous **slime** framework, which consists of four *decoupled components*: (1) the environment server, (2) **PRM / Judge** for reward computation, (3) **Megatron** for policy training, and (4) **SGLang** for policy serving. The environment for personal agents is simply the users' personal devices, which connect to the RL server over HTTP with confidential API keys. The environments for general agents are hosted on cloud services to enable scalable parallelization.

https://arxiv.org/pdf/2603.10165

#Recursive Self Improvement #Continual Learning #LLM-Personalisation

"LLMs don't get better over time the way a human would. The lack of continual learning is a huge huge problem. The LLM baseline at many tasks might be higher than an average human's. But there's no way to give a model high level feedback. You're stuck with the abilities you get out of the box."
– Dwarkesh Patel

https://cameronrwolfe.substack.com/p/rl-continual-learning, https://www.dwarkesh.com/p/timelines-june-2025

Get the Student trained for teaching!

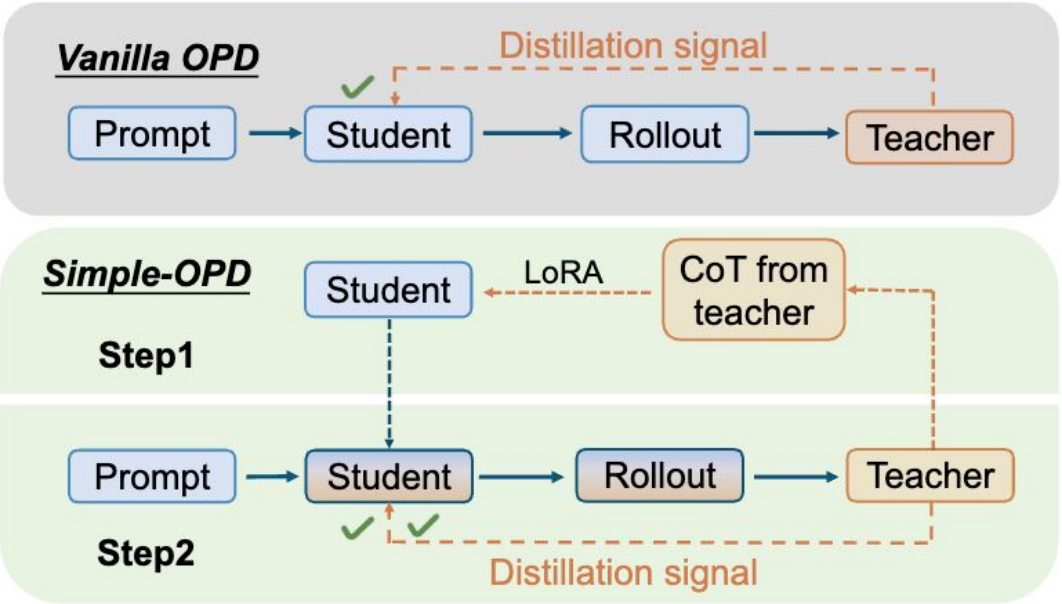
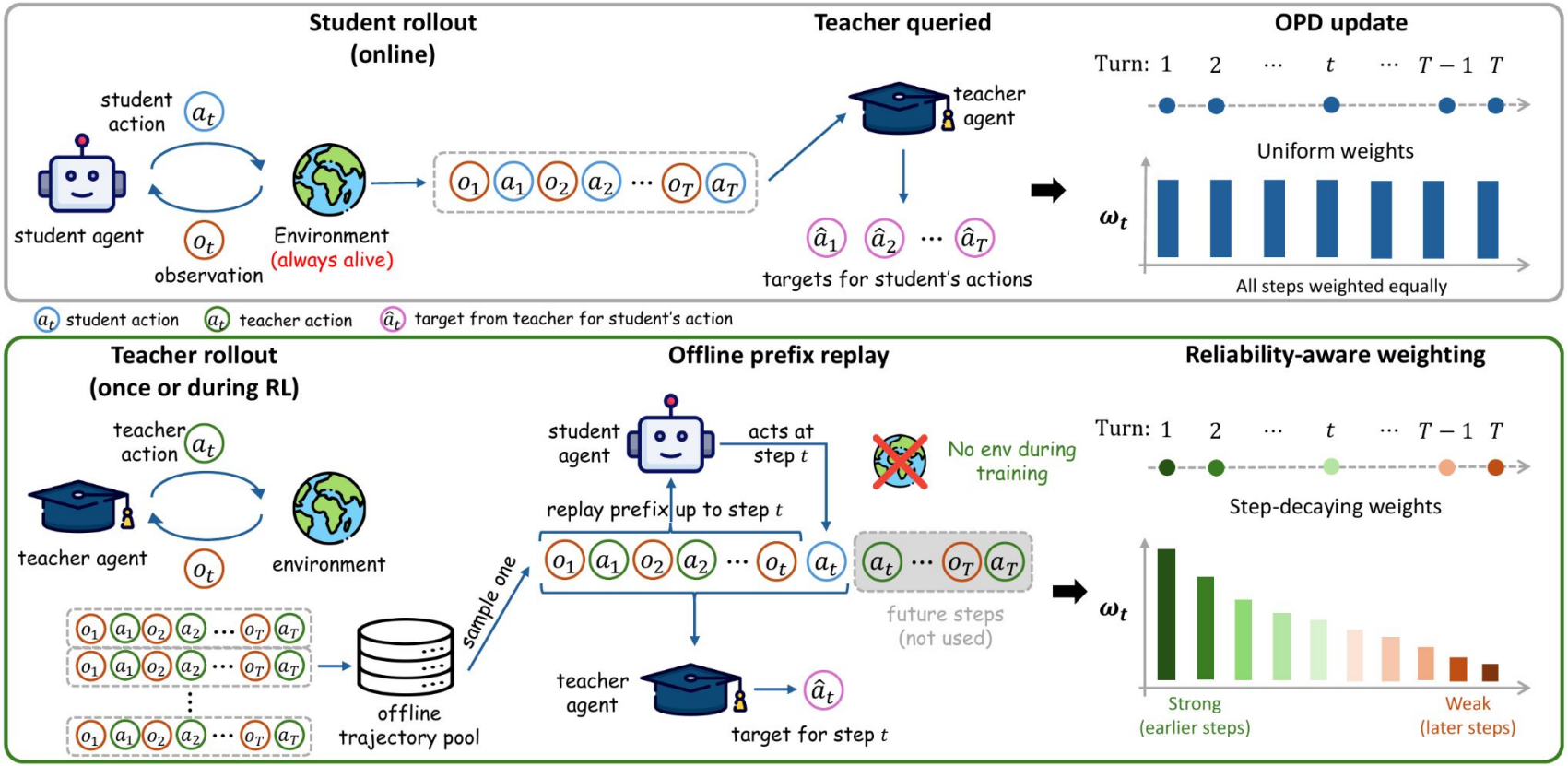


Figure 1: Overview of vanilla OPD and Simple-OPD. Simple-OPD first warms up the student with teacher-generated CoT using LoRA, followed by standard OPD.

https://arxiv.org/pdf/2608.06802

Prefix Replay based OPSD



https://arxiv.org/pdf/2607.04763

SEED

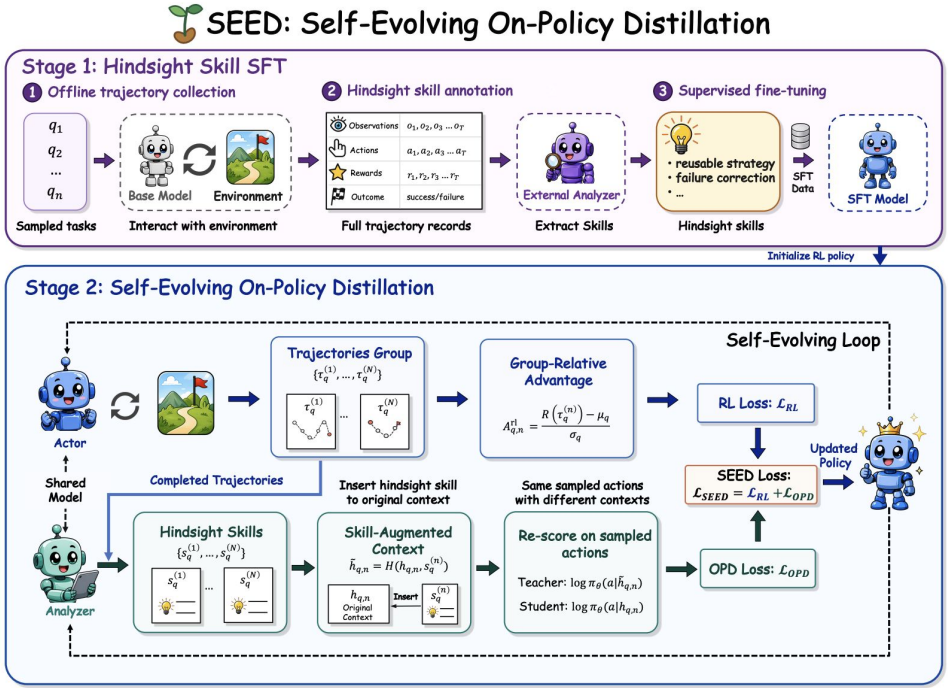


Figure 2: **Overview of SEED. Stage 1** (Hindsight Skill SFT) equips the policy to extract hindsight skills from completed trajectories. **Stage 2** (Self-Evolving On-Policy Distillation) jointly optimizes outcome-based RL and skill-conditioned OPD in a self-evolving agentic loop.

To be direct some strategies are like: [Thoughts!]

- *Should I just let the model interact with the environment and in hindsight based on the recorded interactions, therefore, design foundational skills necessary for the model and then sort of replay those back to model via either SFT or OPSD. Though, SFT, by being off-policy, sound a bit more natural to provide slightly harder and sharper signal, whereas, OPSD, tries to threshold by the teacher's supervision limits of course all that while being on-policy.*

- *Or sort of i model this problem as an adversarial task refinement game, where the Generator keeps generating simulated environments and tasks and the discriminator or critic helps refine the task generation with 'valuable feedback' from the model being trained and simultaneously tested in the environment. On the flavour of Recursive Self Improvement or Continual Learning.*

https://arxiv.org/pdf/2607.14777

Talk Overview

- The LLM Foundations
- Off-Policy vs On-Policy Learning
- The RL Triad: Algorithms, Environments, and Training Infrastructure
  - The Mathematical RL Formulation and Algorithm Zoo
  - Provenance Mapping key to unlocking and sustaining "RL climb"
  - RL Training Infrastructural Resiliency
  - Verifiable & Unverifiable RL Environments
- Fostering Agentic Behaviours via RL
- Multi-Teacher On Policy Distillation
- Additional Open Problems through Practitioner's Lens
- Case Study on Test Time Compute Efficiency
- RL for Safety, Multilingual & Multicultural Environments
- Case Study on recent OpenAI HF Incident

Excellent Example from MAI-Thinking-1, Interplay of RL/Prior stages and Self-Distillation!

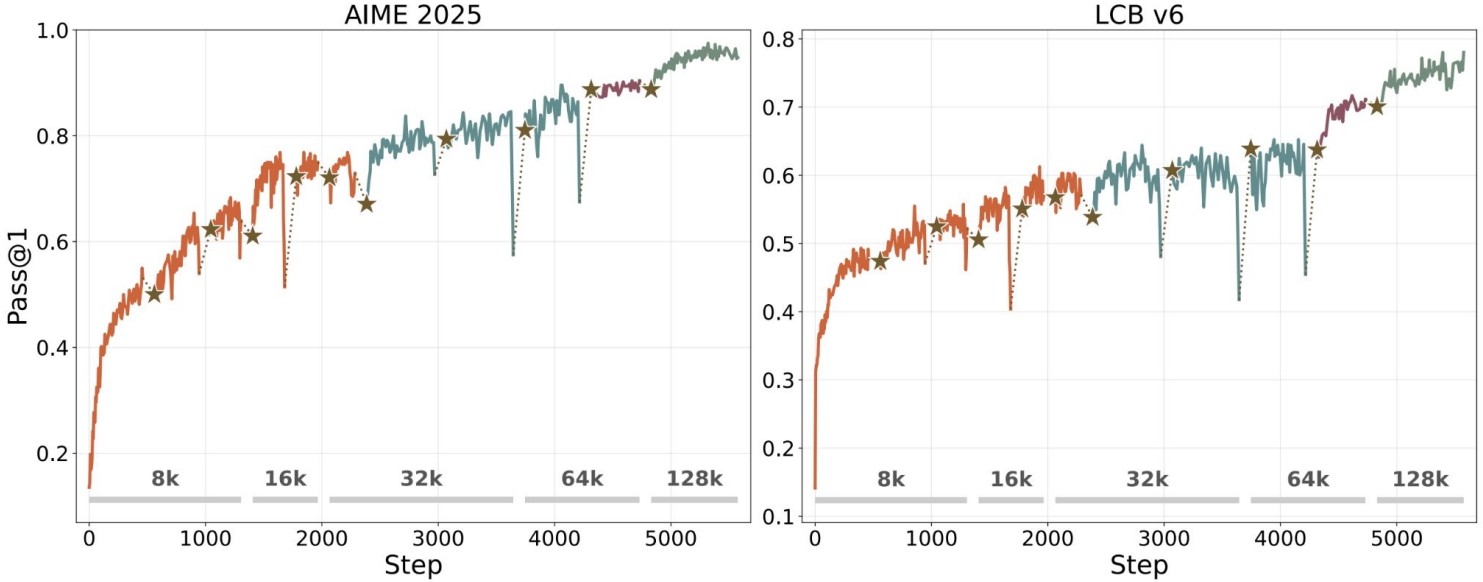


Figure 15. Performance on AIME 2025 (left) and a hard subset of LiveCodeBench v6 (right) during our STEM climb. Self-distillation is indicated through ⋆ markers; different pre- and mid-trained model versions are shown in different colors. Maximum output length throughout training is indicated at the bottom. Self-distillation is an effective way of resetting numerics after a collapse (visible through sudden drops in performance) and changing the base policy during our run. As we made infrastructure and algorithmic improvements throughout our climb, self-distillation because of run collapses became less frequent.

https://microsoft.ai/wp-content/uploads/2026/06/main_20260602_2.pdf

Need for Mid-Training

PRISM: Demystifying Retention and Interaction in Mid-Training

Bharat Runwal, **Ashish Agrawal**, **Anurag Roy**, **Rameswar Panda**

IBM Research, MIT-IBM Watson AI Lab

We present **PRISM**, a comprehensive empirical study of mid-training design choices for large language models (LLMs). Through controlled experiments across seven base models spanning four families (Granite, LLaMA, Mistral, Nemotron-H), two architecture types (dense Transformer and attention-Mamba hybrid), and scales from 3B to 24B parameters, we show that a mid-training phase of ∼**27B** high-quality tokens yields consistent gains of **+15 to +40 points** on math, **+5 to +12 points** on code, and **+6 to +13 points** on science (GPQA-Diamond) benchmarks while preserving general performance. The full PRISM → RL pipeline improves the macro-average (domain-weighted) across six reasoning benchmarks from under 12 to **29–42** (a 3–4× improvement), whereas RL applied directly to most of the base models remains substantially less effective, with AIME scores near zero. Data composition choices matter most at mid-training, not at RL: including science data during mid-training unlocks **+17 to +28 point** GPQA-Diamond gains during RL, while changing the RL mix produces <2 point differences. Mechanistically, mid-training densely restructures >90% of model weights, while RL makes sparse, front-loaded refinements to ∼5% of parameters. Representation analysis (CKA) across three models and three input distributions confirms that RL consistently preserves mid-training's representational geometry (>0.998 CKA) across both dense Transformers and hybrid architectures. Crucially, RL applies identical weight changes regardless of starting point, yet only succeeds on mid-trained models, consistent with mid-training placing the model in a weight configuration from which RL can effectively improve performance. Our results demonstrate that retention-aware mid-training is a highly effective intermediate step for reliable reasoning enhancement and provide practical guidance for designing robust mid-training pipelines.

Correspondence: Bharat Runwal at bharatrunwal@gmail.com

Project Page: Website

Models & Data: HuggingFace

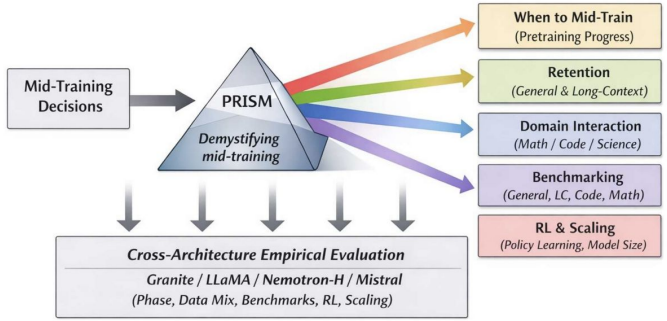


Figure 1 PRISM overview. Mid-training decisions are decomposed into their principal design axes, including retention of general and long-context abilities, domain interaction (math, code, science), benchmark selection, reinforcement learning compatibility, and scaling behavior. PRISM enables holistic evaluation of mid-training choices across model families at scale.

One can actually RL an Open Model and get away!

But in many of the scenarios, having access to the prior-knowledge the model has undergone becomes important.

Hence actually there is indeed a need to build a foundation model for total control on models behaviour.

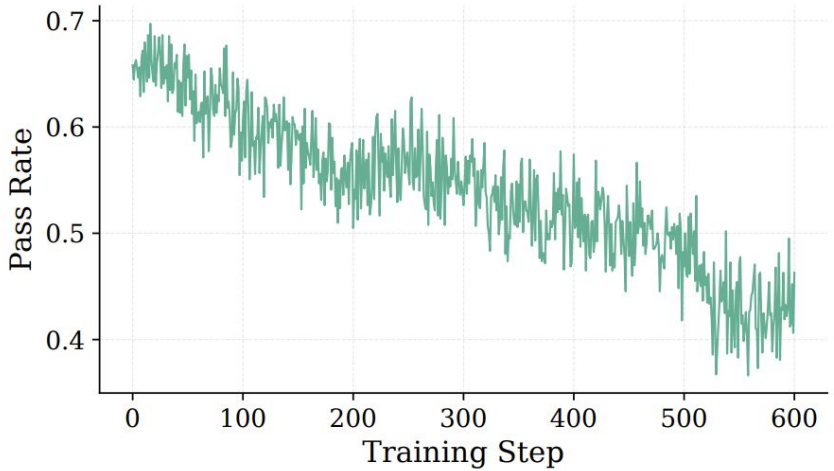
https://arxiv.org/pdf/2603.17074

RL Training Curriculum

Difficulty Calibration: Discussion around RL training medium difficulty problems by removing too hard and too easy problems out of training consideration, and then progressively refining this pursuit of "search" of problems -- he is indicating here of those active sampling based curriculum strategies.

Important relevant references:

- CURRICULUM REINFORCEMENT LEARNING FROM EASY TO HARD TASKS IMPROVES LLM REASONING: https://arxiv.org/pdf/2506.06632
- Nvidia's Nemotron Nano 3's curriculum: https://arxiv.org/pdf/2512.20848



Pass Rate, Pass @ K... Analysis for the RL-ability of the Model



November 25, 2025

Does Reinforcement Learning Really Incentivize Reasoning Capacity in LLMs Beyond the Base Model?

Yang Yue[1*†], **Zhiqi Chen**[1*], **Rui Lu**[1], **Andrew Zhao**[1], **Zhaokai Wang**[2], **Yang Yue**[1], **Shiji Song**[1], **and Gao Huang**[1✉]

[1] LeapLab, Tsinghua University [2] Shanghai Jiao Tong University

[*] Equal Contribution [†] Project Lead [✉] Corresponding Author

Reinforcement Learning with Verifiable Rewards (RLVR) has recently demonstrated notable success in enhancing the reasoning performance of large language models (LLMs), particularly in mathematics and programming tasks. It is widely believed that, similar to how traditional RL helps agents to explore and learn new strategies, RLVR enables LLMs to continuously self-improve, thus acquiring novel reasoning abilities that exceed the capacity of the corresponding base models. In this study, we take a critical look at *the current state of RLVR* by systematically probing the reasoning capability boundaries of RLVR-trained LLMs across various model families, RL algorithms, and math/coding/visual reasoning benchmarks, using pass@k at large k values as the evaluation metric. While RLVR improves sampling efficiency towards correct paths, we surprisingly find that current training *rarely* elicit fundamentally new reasoning patterns. We observe that while RLVR-trained models outperform their base models at smaller values of k (*e.g.*, k=1), base models achieve higher pass@k score when k is large. Moreover, we observe that the reasoning capability boundary of LLMs often narrows as RLVR training progresses. Further coverage and perplexity analysis shows that the reasoning paths generated by RLVR models are already included in the base models' sampling distribution, suggesting that their reasoning abilities originate from and are *bounded* by the base model. From this perspective, treating the base model as an upper bound, our quantitative analysis shows that six popular RLVR algorithms perform similarly and remain far from optimal in fully leveraging the potential of the base model. In contrast, we find that distillation can introduce new reasoning patterns from the teacher and genuinely expand the model's reasoning capabilities. Taken together, our findings suggest that current RLVR methods have not fully realized the potential of RL to elicit genuinely novel reasoning abilities in LLMs. This underscores the need for improved RL paradigms, such as effective exploration mechanism, more deliberate and large-scale data curation, fine-grained process signal, and multi-turn agent interaction, to unlock this potential.

Project Page: https://limit-of-RLVR.github.io

https://arxiv.org/pdf/2504.13837

Pass Rate, Pass @ K... Analysis for the RL-ability of the Model | Techniques to assess RL Readiness of Base/SFT Checkpoint

Olmo 3's Approach

Step 2: offline difficulty filtering As stated previously, to improve the sample efficiency of RL for our reasoner model, we generate eight rollouts for each prompt from the initial checkpoint of the model we train (e.g., if starting from the DPO-trained model, we generate from the DPO checkpoint). We then remove all samples that the model easily solves (that is, those with a pass rate greater than 62.5%). We sample with a temperature of 1.0 and top-p of 1.0, matching how we sample during RL training. We used offline filtering for the 7B OLMO 3 THINK to filter out RL problems that are too easy for our models' training. For the 32B, we rely on active sampling, which fills RL batches only on samples with a non-zero GRPO group gradient, and re-using the 7B DPO-filtered data as the starting point for the model due to compute and time constraints.

MAI-Thinking-1's Approach

Problem sampling. For each problem q in the training set, we generate a group of rollouts from the current inference model. To reduce inference cost, we use an *early exit* strategy: we first sample $G_{\text{early}} < G$ responses and compute their empirical pass rate, i.e. the fraction of responses with positive reward. If the early pass rate lies in an acceptable range $[\rho_{\min}^{\text{early}}, \rho_{\max}^{\text{early}}]$, we sample the full G responses; otherwise the problem is discarded.

After all G responses are generated, we apply a second pass-rate filter $[\rho_{\min}, \rho_{\max}]$ to the full group. Only problems whose full pass rate falls in this range are used for training. The main motivation for this second filter is to remove very low-variance groups: if almost all responses are correct or incorrect, the group provides little useful relative learning signal.

Nvidia's Approach

3.2.2. Data Mixture and Curriculum

We begin by profiling all reinforcement learning (RL) tasks using our supervised fine-tuning (SFT) checkpoint. To focus training on challenging cases, we filter out samples where the SFT checkpoint already achieves a 100% pass rate. We then adopt the curriculum training method introduced in NVIDIA (2025a), which dynamically adjusts task difficulty throughout training.

In each batch, we maintain a fixed ratio of samples across different domains. For each domain, we model the target pass-rate distribution as a Gaussian function, shifting from high pass-rate (easier) samples early in training to low pass-rate (harder) samples later. The target mean of Gaussian distribution decreases linearly throughout training steps. Within each batch, samples from different domains are shuffled. This Gaussian sampling strategy prevents overfitting to either overly easy or overly difficult examples, ensuring a balanced learning progression.

This approach enables a controlled and gradual increase in task difficulty while preserving domain diversity and ensuring efficient batch composition. Figure 6 illustrates how sample difficulty evolves over the course of RL training. Once training progress plateaus, we re-profile the tasks using the best RL checkpoint and construct a new curriculum to further refine performance.

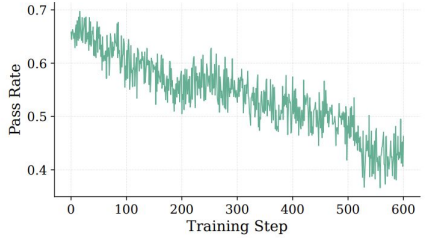


Figure 6 | Batch-wise pass rates across the RL curriculum.

We compare curriculum sampling against random sampling using an intermediate SFT checkpoint, maintaining identical domain ratios in both cases. As shown in Figure 7, curriculum sampling ensures stable learning across multiple domains throughout training. In contrast, random sampling

Olmo3: https://arxiv.org/abs/2512.13961, MAI-Thinking-1: https://microsoft.ai/pdf/mai-thinking-1.pdf, Nvidia Nano: https://arxiv.org/pdf/2512.20848

Pass Rate, Pass @ K... Analysis for the RL-ability of the Model

Nvidia's Approach

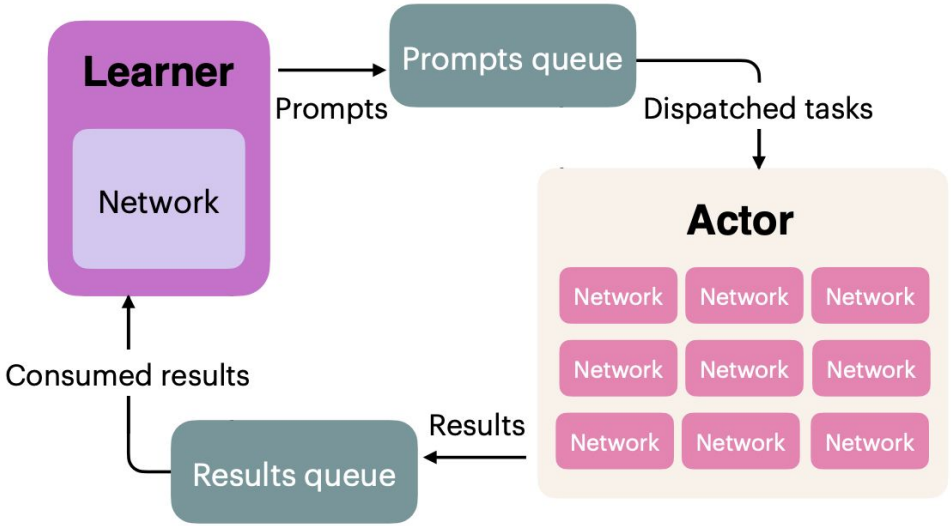
To ensure that the model is adequately challenged, we preprocess the data by independently generating 8 responses per question using LN-Super, calculating the pass rate, and then intentionally discarding prompts with a pass rate of 0.75 or higher, thereby increasing the difficulty of the training data. Besides data filtering, we also find curriculum training to be helpful, as it allows the model to gradually learn from a progression of tasks with increasing difficulty. Specifically, we implement a progressive batching strategy leveraging pre-calculated pass rate as a difficulty metric. Given a fixed batch size, the core of our approach involves dynamically calculating a target distribution of pass rates for each sequential batch. This distribution is modeled using a Gaussian function centered on a difficulty level that progresses from high pass rates (easier examples) for initial batches to low pass rates (harder examples) for later batches. Samples are allocated to each batch primarily based on this target distribution, considering the available count for each pass rate, with any remaining batch capacity filled by prioritizing pass rates with the largest remaining sample pools. This ensures a controlled, gradual increase in average sample difficulty across batches, while samples inside a batch are randomly shuffled. Figure 6 demonstrates the effectiveness of our curriculum strategy, which stabilizes the training process and achieves higher accuracy.

https://arxiv.org/pdf/2505.00949

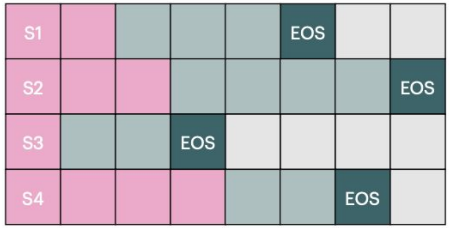
Talk Overview

- The LLM Foundations
- Off-Policy vs On-Policy Learning
- The RL Triad: Algorithms, Environments, and Training Infrastructure
  - The Mathematical RL Formulation and Algorithm Zoo
  - Provenance Mapping key to unlocking and sustaining "RL climb"
  - RL Training Infrastructural Resiliency
  - Verifiable & Unverifiable RL Environments
- Fostering Agentic Behaviours via RL
- Multi-Teacher On Policy Distillation
- Additional Open Problems through Practitioner's Lens
- Case Study on Test Time Compute Efficiency
- RL for Safety, Multilingual & Multicultural Environments
- Case Study on recent OpenAI HF Incident

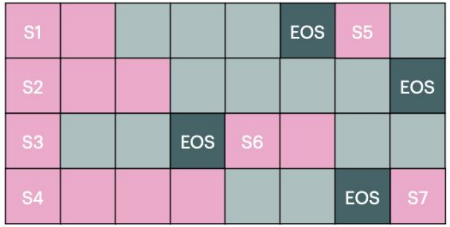
RL Training Infrastructure



(a) Distributed reinforcement learning architecture



(b) Static batching



(c) Continuous batching

Figure 17 Overview of OlmoRL infrastructure. On the left: distributed reinforcement learning architecture (Figure 17a). On the right: static vs. continuous batching. Static batching (Figure 17b) wastes compute when generations have variable sequence lengths. Pink cells are prefilled tokens, green cells are decoded tokens, with dark green representing EOS. Grey indicates that sequence is not doing anything, so continuous batching (Figure 17c) backfills finished rows immediately, resulting in no wasted compute.

https://arxiv.org/pdf/2512.13961

Case Study: MAI-Thinking-1, RL Training Infra

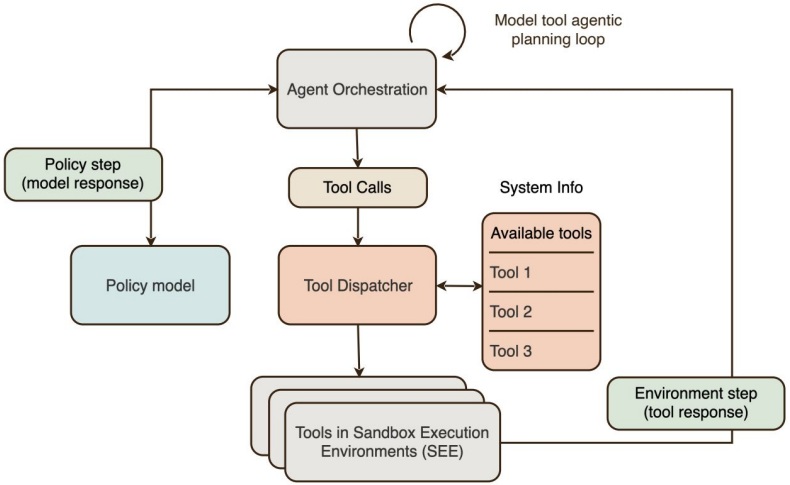


Figure 18. Agentic loop and multi-step orchestration in RL training.

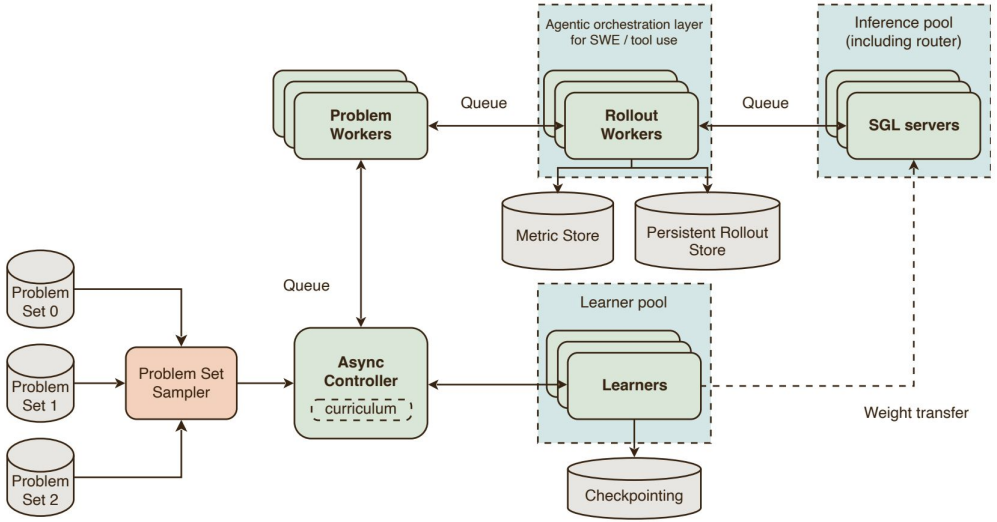


Figure 19. An overview of Rocket, our large scale Reinforcement Learning framework.

https://microsoft.ai/wp-content/uploads/2026/06/main_20260602_2.pdf

RL Training Infrastructure

On-policy vs. off-policy

Ideal / Theory (on-policy): generate → update → generate → update

Each batch of completions is scored and used for a short update window (one or a few epochs), then discarded.

Reality (async): generation and training overlap on different GPU groups for better throughput. The model used for generation may be 1–N steps behind the training model.

Tradeoff: perfect on-policy is slow (GPUs idle during generation or training). Slight staleness is usually fine.

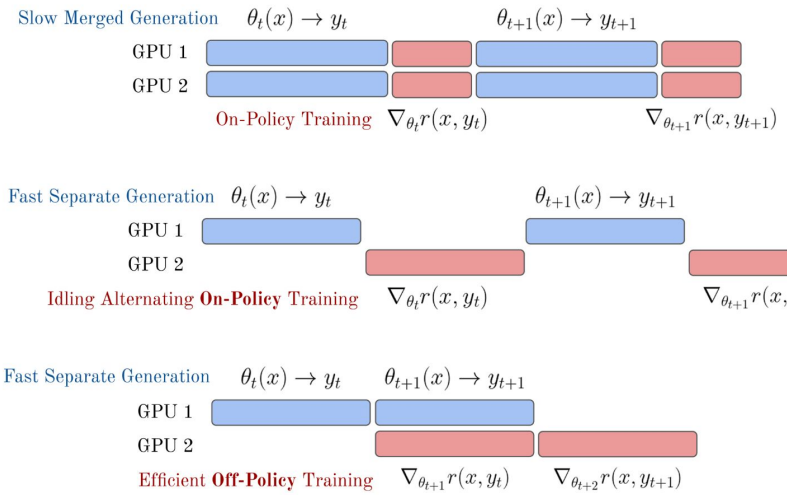
Challenges: *Resiliency between the Learner Module and the Roll-out Cluster, Accumulation buffer, Off-policy tradeoffs while still being On policy (how on-policy RL Is secretly off-policy), in-flight gradient transfer, Delayed Gradient Calculation, Lazy KV Cache update*

Synchronous vs. asynchronous training

Your On-policy RL is secretly Off-policy!

Delayed Grad Updates

Active Sampling vs Fixed Len Dynamic Sampling



https://arxiv.org/pdf/2512.13961

Case Study: Olmo-RL, 2025

We make the following improvements[34] over vanilla GRPO:

- **Zero gradient signal filtering**: We remove groups of instances whose rewards are all identical (i.e., a batch with zero standard deviation in their advantage) to avoid training on samples that provide zero gradient, similar to DAPO (Yu et al., 2025).
- **Active sampling**: We maintain a consistent batch size in spite of zero gradient filtering with a novel, more efficient version of dynamic sampling (Yu et al., 2025), see Section §4.4.3 for details.
- **Token-level loss**: We use a token-level loss to normalize the loss by the total number of tokens across the batch (Yu et al., 2025), rather than per-sample to avoid a length bias.
- **No KL loss**: We remove the KL loss as a common practice (GLM-4.5 Team et al., 2025; Yu et al., 2025; Liu et al., 2025b) as it allows less-restricted policy updates, and removing it does not lead to over-optimization or destabilized training.
- **Clip higher**: We set the upper-bound clipping term in the loss to a slightly higher value than the lower bound to enable larger updates on tokens, as proposed by Yu et al. (2025).
- **Truncated importance sampling**: To adjust for differences between log probabilities from the inference and training engines, we multiply the loss by the truncated importance sampling ratio, following Yao et al. (2025).
- **No standard deviation normalization**: When calculating advantage, we do not normalize by the standard deviation of the group, following Liu et al. (2025b). This removes a difficulty bias, where questions with low standard deviation in their rewards (e.g., too hard or too easy) have their advantages significantly increased by the normalization term.

OlmoRL formulation Our final objective function includes a token-level loss, truncated importance sampling, clip-higher, and no standard deviation in the advantage calculation:

$$\mathcal{J}(\theta) = \frac{1}{\sum_{i=1}^{G}|y_i|}\sum_{i=1}^{G}\sum_{t=1}^{|y_i|}\min\left(\frac{\pi\left(y_{i,t} \mid x, y_{i,<t}; \theta_{\text{old}}\right)}{\pi_{\text{vllm}}\left(y_{i,t} \mid x, y_{i,<t}; \theta_{\text{old}}\right)}, \rho\right)\min\left(r_{i,t}\,A_{i,t},\ \text{clip}\left(r_{i,t},\ 1-\varepsilon_{\text{low}},\ 1+\varepsilon_{\text{high}}\right)A_{i,t}\right), \quad (1)$$

where $r_{i,t} = \frac{\pi(y_{i,t}|x,y_{i,<t};\theta)}{\pi(y_{i,t}|x,y_{i,<t};\theta_{\text{old}})}$, ε_{low} and $\varepsilon_{\text{high}}$ are the clipping hyperparameters. Here, $y_i \sim \pi_{\text{vllm}}(\cdot \mid x; \theta_{\text{old}})$ and $\pi_{\text{vllm}}(\cdot \mid x; \theta_{\text{old}})$ are the token probabilities returned from vLLM, ρ is the truncated importance sampling cap value (Yao et al., 2025), and the advantage $A_{i,t}$ for the t-th token t in the response y_i is calculated within the group G based on the relative reward of the outputs inside each group:

$$A_{i,t} = \left(r\left(x, y_i\right) - \text{mean}\left(\{r\left(x, y_i\right)\}_{i=1}^{G}\right)\right). \quad (2)$$

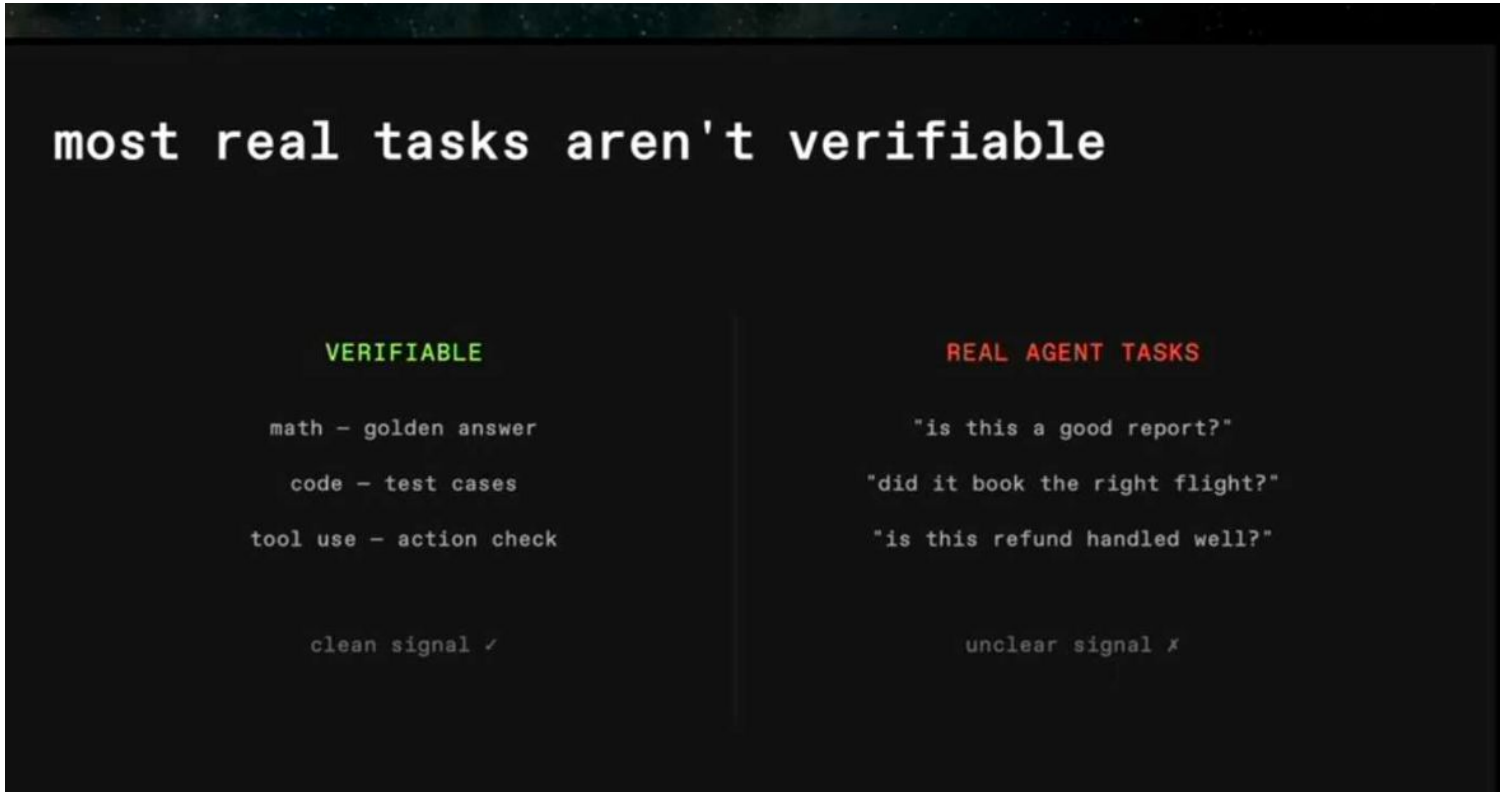
$r\left(x, y_i\right)$ is the reward score returned by the corresponding verifier. Our hyperparameters for various runs are in Appendix Table 49.

https://arxiv.org/pdf/2512.13961

Talk Overview

- The LLM Foundations
- Off-Policy vs On-Policy Learning
- The RL Triad: Algorithms, Environments, and Training Infrastructure
  - The Mathematical RL Formulation and Algorithm Zoo
  - Provenance Mapping key to unlocking and sustaining "RL climb"
  - RL Training Infrastructural Resiliency
  - Verifiable & Unverifiable RL Environments
- Fostering Agentic Behaviours via RL
- Multi-Teacher On Policy Distillation
- Additional Open Problems through Practitioner's Lens
- Case Study on Test Time Compute Efficiency
- RL for Safety, Multilingual & Multicultural Environments
- Case Study on recent OpenAI HF Incident

RL Training Environments: Verifiable Domains and Unverifiable Domains



Reinforcement Learning without Verifiable Rewards — Will Brown, Prime Intellect: https://www.youtube.com/watch?v=AQv3qRCG6Gw

Talk Overview

The LLM Foundations

Off-Policy vs On-Policy Learning

The RL Triad: Algorithms, Environments, and Training Infrastructure

- The Mathematical RL Formulation and Algorithm Zoo
- Provenance Mapping key to unlocking and sustaining "RL climb"
- RL Training Infrastructural Resiliency
- Verifiable & Unverifiable RL Environments

Fostering Agentic Behaviours via RL

Multi-Teacher On Policy Distillation

Additional Open Problems through Practitioner's Lens

Case Study on Test Time Compute Efficiency

RL for Safety, Multilingual & Multicultural Environments

Case Study on recent OpenAI HF Incident

Fostering Agentic Behaviours via RL



Reinforcement Learning without Verifiable Rewards — Will Brown, Prime Intellect: https://www.youtube.com/watch?v=AQv3qRCG6Gw

Frontier Agentic Reinforcement Learning for Long-Horizon Software Engineering and Autonomous LLM Agents

Background of the Problem

- The emergence of Agentic Reinforcement Learning (Agentic RL) represents a fundamental evolution in the post-training of Large Language Models (LLMs), extending them beyond passive next-token prediction toward autonomous systems capable of reasoning, planning, decision making, tool use, and interaction with dynamic environments.
- In conventional LLM reinforcement learning, the model typically produces a response and receives a reward based primarily on the quality of that response. Agentic RL instead treats the model as an interactive policy operating over an environment, where successful behavior may require executing a long sequence of actions, observations, reasoning steps, and decisions before achieving a desired outcome.
- An agent trained through Agentic RL may dynamically invoke tools such as code interpreters, search engines, browsers, APIs, databases, and operating-system interfaces, observe the resulting environment state, revise its strategy, and continue acting until a task is completed. This creates a fundamentally different learning problem from single-turn response optimization.
- A central characteristic of Agentic RL is long-horizon decision making. The reward may be sparse or delayed, with success depending on a chain of interdependent actions rather than any individual response. Consequently, Agentic RL introduces challenges such as trajectory-level credit assignment, exploration, multi-turn reasoning, tool selection, error recovery, planning, and learning from environmental feedback.
- Agentic RL therefore combines language modeling with interactive reinforcement learning, enabling LLMs to learn not only what to say, but what to do, when to act, which tools to use, how to adapt to feedback, and how to recover from failures while pursuing long-term objectives. This paradigm is increasingly relevant to Software Engineering Agents, Coding Agents, Search Agents, Computer-Use Agents, Cybersecurity Agents, and other autonomous systems operating in complex environments.
- As LLMs evolve into increasingly capable agents, Agentic RL provides a framework for optimizing these systems directly on task outcomes and real-world interaction trajectories, making it a key direction for developing models capable of reliable, autonomous, and long-horizon problem solving.

Core Area: Foundations of Reinforcement Learning for Post-Training Large Language Models (LLMs); Agentic Reinforcement Learning, Coding Agents and Software Engineering Agents

Frontier Agentic Reinforcement Learning for Long-Horizon Software Engineering and Autonomous LLM Agents

- **Representative Benchmarks:**
  a. **General Agent:** BFCL, Tau Bench v3 (Airline, Retail, Telecom, Banking), AceBench, LitQA2, VITA Bench, Deep Planning, Tool Decathlon, MCP Mark, MCPAtlas, BIRD Bench, GDPVal, WildClawBench, ClawBench, PinchBench, ClawEval; Search Agent: HLE* (w & w/o Tools), Browse Comp (with search), Wide Search, SimpleQA* (w & w/o Tools);
  b. **Coding Agents:** Terminal Bench, SWE-Bench, SWE-Bench-Pro, SWE-Agentless, SecCodeBench; Advanced Benchmarks inlcuding: FrontierCode (Diamond), GDP-ValAA, AutomationBench, OS-World, Humanities Last Exam (HLE), BioMysteryBench, ExploitBench, Health Bench (Professional), KernelBench, BrowseComp, YC-Bench, OfficeQA Pro, Spreadsheet Bench, VIBE-V2, LiveSQLBench, PostTrainBench, PaperBench, DRACO; Cyber Security Specific Benchmarks: Cybench, BountyBench, InterCode-CTF, NYU CTF Bench, CVE-Bench, CIBER, CyBiasBench; etc.

- **Inspiration/Motivating Examples:**
  a. Strong Agents (typically a Model+Harness): Fable 5 (& Mythos), Opus 4.8, GPT-5.6 Sol, Kimi K3, GLM 5.2, MiniMax M3, Composer 2.5, Muse-Spark 1.1, Grok 4.5, Qwen-3.6 Plus and under Harnesses Including but not limited to: Claude Code, Codex, OpenCode, Qoder, Qwen Code, OpenCode, OpenHands, MiniSWE Agent, GLM-Z.AI, Kimi-Harness, Nous Hermes Agent etc.

Core Area: Foundations of Reinforcement Learning for Post-Training Large Language Models (LLMs); Agentic Reinforcement Learning, Coding Agents and Software Engineering Agents

Case Study: MAI's Agentic RL Infrastructure

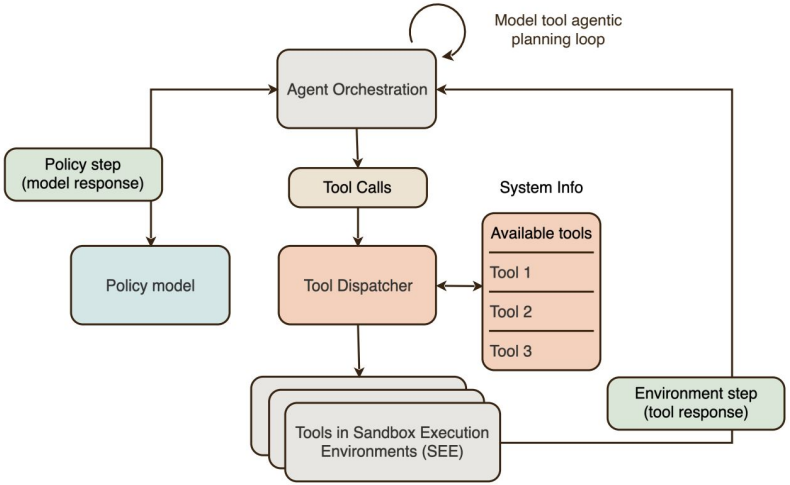


Figure 18. Agentic loop and multi-step orchestration in RL training.

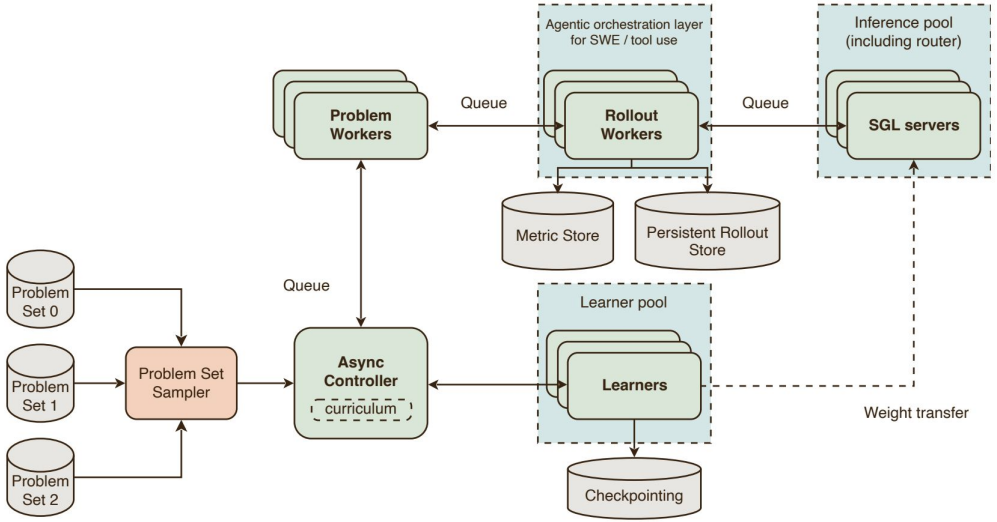


Figure 19. An overview of Rocket, our large scale Reinforcement Learning framework.

Reference: https://microsoft.ai/pdf/mai-thinking-1.pdf

Core Area: Foundations of Reinforcement Learning for Post-Training Large Language Models (LLMs); Agentic Reinforcement Learning, Coding Agents and Software Engineering Agents

Frontier Agentic Reinforcement Learning for Long-Horizon Software Engineering and Autonomous LLM Agents

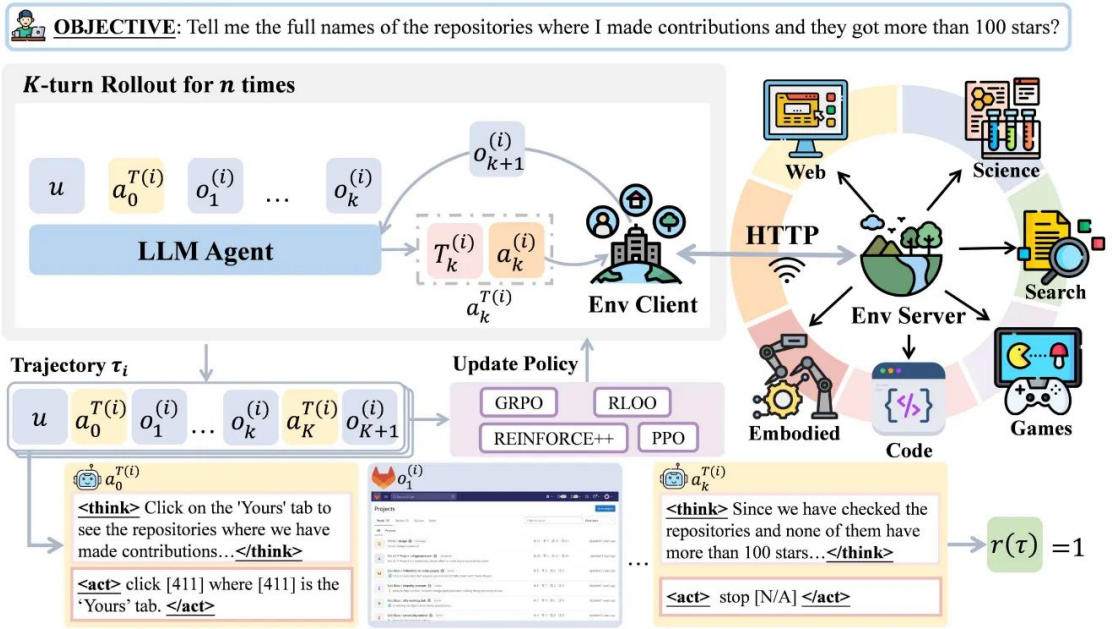


Figure 2 Overview of the AgentGym-RL framework. It features a decoupled, flexible, and extensible architecture, comprising three primary modules—the environment, the agent, and the training module. It supports diverse scenarios, environments, and algorithms.

Reference: https://cameronrwolfe.substack.com/p/agentic-rl

Core Area: Foundations of Reinforcement Learning for Post-Training Large Language Models (LLMs); Agentic Reinforcement Learning, Coding Agents and Software Engineering Agents

Case Study: ECHO: Terminal Agents Learn World Models for Free

ECHO: Terminal Agents Learn World Models for Free https://arxiv.org/pdf/2605.24517

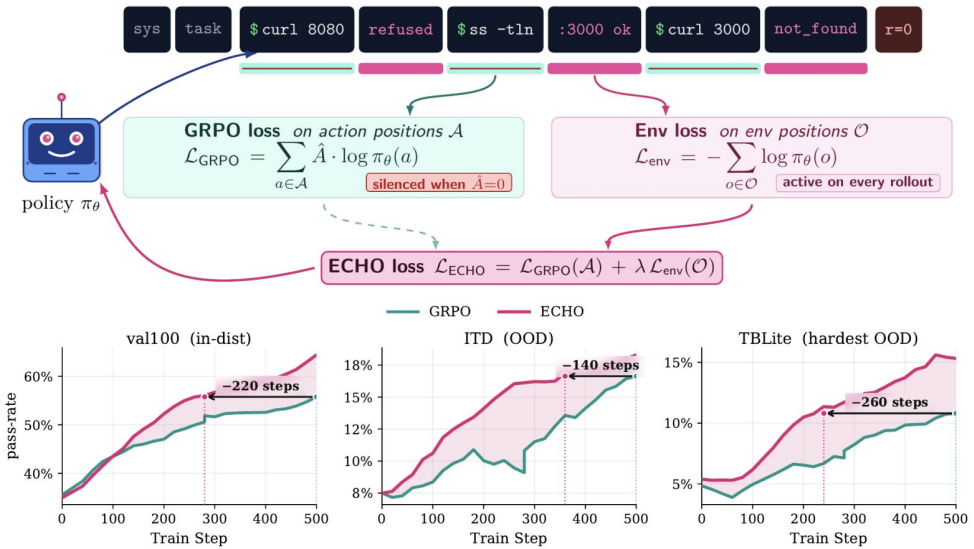


Figure 1: ECHO turns terminal feedback into supervision during agent RL. A terminal-agent rollout interleaves assistant actions, with environment observations. In standard GRPO, previous terminal outputs can inform future commands, but loss is applied only to action-token positions and is driven by sparse outcome-level rewards. ECHO adds a complementary cross-entropy loss on environment-observation tokens, training the same policy to predict the terminal responses caused by its own actions. ECHO reuses the same rollout and forward pass while making every terminal response an additional training target. Bottom: across held-out evaluations, ECHO improves task solve rate relative to matched GRPO-only runs and also reaches the peak GRPO performance substantially earlier in training.

Typical Agentic trajectory template will look like this: SYSTEM PROMPT -> TASK PROMPT -> LLM_Thinking_action_1 -> TOOL CALLS_1 -> TOOL RESULTS_1 -> LLM_Thinking_action_2 -> TOOL CALLS_2 -> TOOL RESULTS_2 -> SO ON TILL TASK EXECUTION COMPLETES

Now in traditional GRPO, while computing its loss, we will ideally drop the TOOL RESULTS outputs that are returned by external environments; and instead we will only assign a typical sparse reward, indicating whether the above long-horizon set of action-reactions based interaction with the environment was successful in completing the task or not. Now of course there are certain variants where process level rewards are also applied, but essentially there also we will drop the results returned from the environments. So what they are saying is that you apply SFT loss on that as well thereby making every environment/terminal response an additional training target.

Essentially what is happening here is, the application of SFT's CE loss over the observations from the environment, alongside standard GRPO loss, there by encourages the LLM to maximise these rewards, by taking actions that are anticipated to lead successful tool executions over the long horizon interaction with the environment. --> "Model is trying to learn to take up more appropriate actions"

Talk Overview

The LLM Foundations

Off-Policy vs On-Policy Learning

The RL Triad: Algorithms, Environments, and Training Infrastructure

- The Mathematical RL Formulation and Algorithm Zoo
- Provenance Mapping key to unlocking and sustaining "RL climb"
- RL Training Infrastructural Resiliency
- Verifiable & Unverifiable RL Environments

Fostering Agentic Behaviours via RL

Multi-Teacher On Policy Distillation

Additional Open Problems through Practitioner's Lens

Case Study on Test Time Compute Efficiency

RL for Safety, Multilingual & Multicultural Environments

Case Study on recent OpenAI HF Incident

Multi-Teacher On-Policy Distillation for Reinforcement Learning-based Capability Composition in Large Language Models

Background of the Problem

- The recent progress of Reinforcement Learning (RL) has demonstrated that Large Language Models (LLMs) can acquire increasingly sophisticated capabilities, including mathematical reasoning, coding, scientific problem solving, long-horizon planning, tool use, and autonomous agentic behaviour. As these capabilities become increasingly specialized, a natural paradigm is to train expert policies that are individually optimized for different domains or tasks and subsequently transfer their capabilities into a unified foundation model.
- This paradigm introduces a fundamental challenge in capability composition: how can a single student model effectively acquire and integrate capabilities learned independently by multiple expert teachers? Unlike conventional knowledge distillation, where a student typically learns from a single teacher or a relatively homogeneous supervision signal, multi-expert distillation involves heterogeneous teachers with different capabilities, behaviours, reward structures, optimization histories, model architectures, tokenizers, and parameter scales. The central challenge is therefore not merely improving the reinforcement learning objective, but understanding the mechanisms that govern capability transfer and composition. A student may successfully acquire one capability while struggling to absorb another, or may experience interference and degradation when multiple capabilities are learned simultaneously. This makes the student's learning dynamics, capacity to absorb new capabilities, and ability to retain previously acquired knowledge central to the problem.
- Capability composition can consequently be viewed as a continual learning and optimization problem in which multiple expert policies act as sources of complementary knowledge. Effective transfer may depend on factors such as the competence of individual teachers, the ordering and difficulty of acquired capabilities, the interaction between heterogeneous objectives, the learning dynamics of the student, and the mechanisms used to select or combine teacher-generated trajectories. This gives rise to several fundamental questions: How should knowledge from multiple experts be combined without causing cross-domain interference? How should teachers be selected or routed based on their competence and the student's current capabilities? How much new information can a student effectively absorb at a given stage of training? How can newly acquired capabilities be retained during subsequent distillation? And how does the choice of teacher-student interaction and optimization strategy affect the efficiency and quality of capability transfer?
- More broadly, multi-expert RL distillation seeks to move beyond training increasingly capable models through scale alone, toward systematically composing specialized capabilities into a single general-purpose model. Understanding this capability transfer process is an important step toward building foundation models that can continuously acquire, integrate, and retain diverse forms of intelligence from multiple specialized experts.

Core Area: Foundations of Reinforcement Learning for Post-Training Large Language Models (LLMs); Multi-Teacher Distillation; Capability Composition; Continual Learning, Federated Learning.

Case Study: Multi-Teacher On Policy Distillation, MOPD

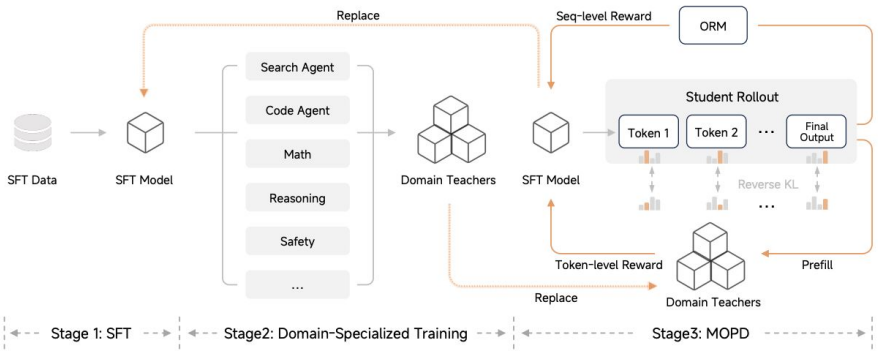


Figure 3 Overview of MiMo-V2-Flash post-training stages.

Stage 1: Supervised Fine-Tuning (SFT) We establish foundational instruction-following capabilities through supervised learning on high-quality instruction-response pairs, enabling the model to understand and execute user requests across diverse domains.

Stage 2: Domain-Specialized Training We train a suite of domain-specialized teacher models through independent RL optimization on focused tasks including agentic capabilities (search, coding, general tool use) and non-agentic tasks (mathematical reasoning, general reasoning, safety alignment). Each teacher achieves superior performance in its respective domain through targeted optimization with domain-specific reward signals.

Stage 3: Multi-Teacher On-Policy Distillation Rather than merging model parameters or generating static offline datasets from experts, we formulate multi-teacher knowledge integration as an on-policy reinforcement learning process. The student model samples from its own evolving distribution and receives token-level supervision from domain-specific teachers through KL divergence rewards (Agarwal et al., 2023; Gu et al., 2024c; Lu and Lab, 2025), effectively combining specialized capabilities without the traditional trade-offs (Table 7).

https://arxiv.org/pdf/2601.02780, https://arxiv.org/pdf/2606.30406v1

4.4 Technical Formulation of MOPD

Having established the foundation through SFT and trained specialized teachers through domain-specific RL, we now formalize the multi-teacher on-policy distillation mechanism that integrates these specialized capabilities into a unified student model.

Specifically, we cast multi-teacher distillation as an on-policy reinforcement learning objective. Let π_θ denote the target student policy optimized in the training engine, μ_θ denote the student sampling policy adopted in the inference engine, and π_{domain_x} denote the teacher policy specialized for the domain of prompt x sampled from distribution $\mathcal{D}$. Let $\text{sg}[\cdot]$ denote the stop-gradient operator. The reverse KL divergence loss between student and teacher is defined as:

$$\mathcal{L}_{\text{reverse-KL}}(\theta) = -\mathbb{E}_{x\sim\mathcal{D},y_t\sim\pi_\theta(\cdot|x,y_{<t})} \log \frac{\pi_{\text{domain}_x}(y_t|x,y_{<t})}{\pi_\theta(y_t|x,y_{<t})}. \tag{5}$$

The gradient is:

$$\nabla_\theta \mathcal{L}_{\text{reverse-KL}}(\theta) = -\mathbb{E}_{x\sim\mathcal{D},y_t\sim\pi_\theta(\cdot|x,y_{<t})} \left[\log \frac{\pi_{\text{domain}_x}(y_t|x,y_{<t})}{\pi_\theta(y_t|x,y_{<t})} \nabla_\theta \log \pi_\theta(y_t|x,y_{<t})\right]. \tag{6}$$

Following Zhao et al. (2025), we apply training-inference importance sampling and discard tokens that exhibit large discrepancies. We then define the surrogate loss of MOPD as:

$$\mathcal{L}_{\text{MOPD}}(\theta) = -\mathbb{E}_{x\sim\mathcal{D},y\sim\mu_\theta(\cdot|x)} \left[\frac{1}{|y|}\sum_{t=1}^{|y|} w_t \hat{A}_{\text{MOPD},t} \log \pi_\theta(y_t|x,y_{<t})\right], \tag{7}$$

where

$$w_t(\theta) = \begin{cases} \text{sg}\left[\frac{\pi_\theta(y_t|x,y_{<t})}{\mu_\theta(y_t|x,y_{<t})}\right], & \epsilon_{\text{low}} \le \frac{\pi_\theta(y_t|x,y_{<t})}{\mu_\theta(y_t|x,y_{<t})} \le \epsilon_{\text{high}}, \\ 0, & \text{other}, \end{cases} \qquad \hat{A}_{\text{MOPD},t} = \text{sg}\left[\log \frac{\pi_{\text{domain}_x}(y_t|x,y_{<t})}{\pi_\theta(y_t|x,y_{<t})}\right]. \tag{8}$$

By default, we combine the advantages of MOPD with other types of advantages, such as those computed using Outcome Reward Models (ORMs), including GRPO (Shao et al., 2024). Let $\hat{A}_{\text{ORM}}$ denote the advantages computed by the ORMs; the final advantages are given by:

$$\hat{A}_{\text{MOPD},t} = \text{sg}\left[\log \frac{\pi_{\text{domain}_x}(y_t|x,y_{<t})}{\pi_\theta(y_t|x,y_{<t})}\right] + \alpha \hat{A}_{\text{ORM}}. \tag{9}$$

Figure 6 demonstrates the effectiveness of MOPD compared to traditional post-training approaches. On mathematical reasoning (AIME 2025) and coding (LiveCodeBench) benchmarks, MOPD successfully preserves and combines specialized capabilities from multiple teachers, achieving performance that matches or exceeds the strongest teacher in most domains.

Case Study: Multi-Teacher On Policy Distillation, MOPD

Table 2 Qwen3-30B-A3B: multi-domain capability integration. Per-benchmark columns are accuracy (%); higher is better. "RL Teacher" is the Stage-2 specialised RL teacher for that domain. The **Norm. score** column is the normalised score defined in §4.1; higher is better. Within the six integration methods we **bold** the column best and underline the runner-up.

	Math		Instruction Following		SWE	
Method	AIME25	AIME26	IFBench	IFEval	SWE-bench Verified	Norm. score
Student (SFT-only)	45.42	54.48	42.69	84.17	35.80	0.0000
RL Teacher	54.79	63.65	78.40	95.50	51.20	1.0000
Mix-RL	**52.71**	63.75	75.00	94.58	48.80	0.8818
Cascade RL	48.54	61.88	77.11	95.80	47.80	0.7752
Off-Policy Finetune	51.56	63.44	**80.95**	93.35	45.80	0.8241
Param-Merge (Avg.)	47.81	59.58	53.74	88.79	39.60	0.3280
Param-Merge (Task Arith.)	49.38	63.96	78.23	**95.81**	48.80	0.8574
MOPD (ours)	51.46	**65.31**	77.89	93.84	**50.40**	**0.9373**
Δ(MOPD − RL Teacher)	−3.33	+1.66	−0.51	−1.66	−0.80	−0.0627

https://arxiv.org/pdf/2601.02780, https://arxiv.org/pdf/2606.30406v1

Multi-Teacher On Policy Distillation, MOPD

MOPD Paper Findings:

1. Mix-RL, Cascade RL, and Off-Policy Finetune each leave domain-specific gaps, in different shapes.
2. MOPD achieves the highest normalised score with the most uniform profile.
3. Param-Merge is highly sensitive to the merging recipe.
4. MOPD is markedly more sample-efficient.
5. Policy-gradient and top-k distillation perform comparably
6. Same-origin teachers stabilize distillation
7. Multi-round student–teacher evolution

https://arxiv.org/pdf/2601.02780

Multi-Teacher On-Policy Distillation for Reinforcement Learning-based Capability Composition in Large Language Models #Research Direction

Potential Open Problems

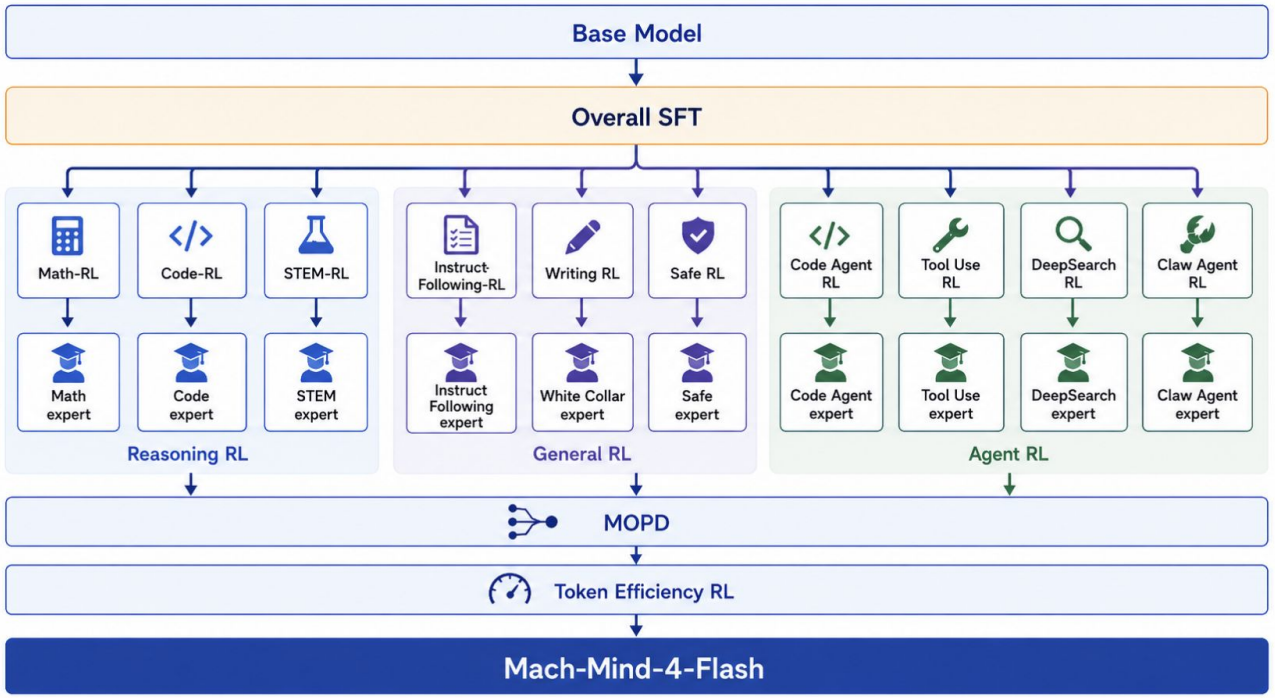
- **Capability-aware Curriculum Learning:** Adaptive data scheduling across heterogeneous expert domains, Conflict-aware sampling strategies, Difficulty-aware and competence-aware curricula, Capability enrichment through optimal task sequencing;
- **Student Competence and Capability Acquisition:** Measuring the student's absorbing power, Teacher–student capability gap analysis, Representation capacity versus transfer efficiency, Capability acquisition scaling laws;
- **Reasoning Depth:** Reasoning Budget decision, low, max and high thinking; Teacher Selection and Routing: Adaptive teacher routing policies, Confidence-aware teacher selection, Mixture-of-teachers optimization, Dynamic teacher weighting during RL training;
- **Training Algorithm Design:** Multi-teacher policy optimization, Adaptive KL regularization, Policy and value distillation, Frozen-teacher versus simultaneous optimization, Local-global iterative optimization strategies;
- **Continual Capability Learning:** Catastrophic forgetting mitigation, Replay-based capability preservation, Optimal context selection, Continual reinforcement learning;
- **Multi-Objective Reinforcement Learning:** Heterogeneous reward optimization, Capability trade-off analysis, Pareto-efficient policy optimization, Reward balancing strategies;
- **Scaling and Systems Analysis:** Number of global distillation rounds, Communication versus capability gains, FLOP-efficient distillation, Teacher synchronization strategies, Broadcast versus aggregate paradigms;
- **Evaluation and Benchmarking:** Capability transfer metrics, Forgetting metrics, Transfer efficiency, Distillation headroom, Cross-domain interference analysis.

Multi-Teacher On-Policy Distillation for Reinforcement Learning-based Capability Composition in Large Language Models

Core Problem to Solve: To investigate the fundamental principles governing capability transfer from multiple Reinforcement Learning (RL) expert teachers into a unified Large Language Model. How can one go about developing principled algorithms for curriculum learning, teacher selection, student competence estimation, and continual capability acquisition that maximize knowledge transfer while minimizing capability interference, catastrophic forgetting, and computational cost.

Simply put: ***"How do we distill all the knowledge from the expert models into the student so that it leaves the least headroom or it beats it in terms of the teachers knowledge?"***

Multi-Teacher On-Policy Distillation for Reinforcement Learning-based Capability Composition in Large Language Models



Reference: https://www.alphaxiv.org/abs/2607.09375

Core Area: Foundations of Reinforcement Learning for Post-Training Large Language Models (LLMs); Multi-Teacher Distillation; Capability Composition; Continual Learning, Federated Learning.

Multi-Teacher On-Policy Distillation for Reinforcement Learning-based Capability Composition in Large Language Models

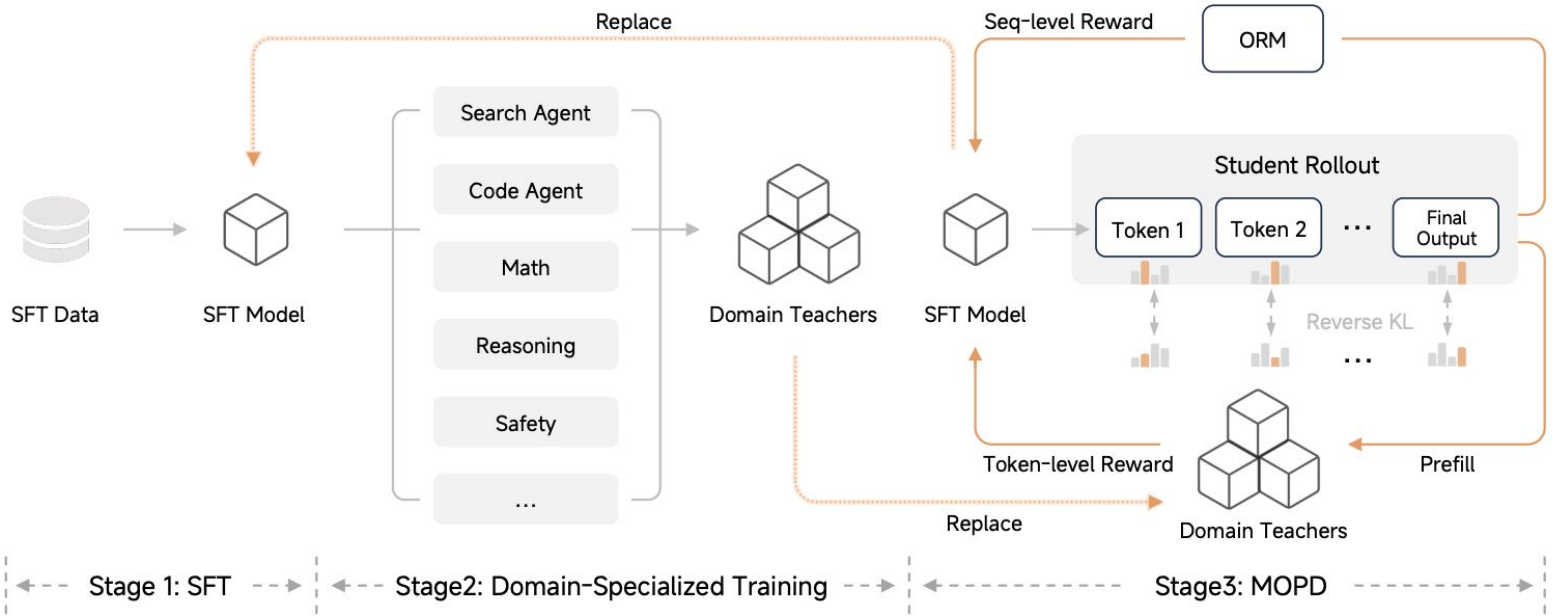


Figure 3 Overview of MiMo-V2-Flash post-training stages.

Reference: https://arxiv.org/pdf/2601.02780

Core Area: Foundations of Reinforcement Learning for Post-Training Large Language Models (LLMs); Multi-Teacher Distillation; Capability Composition; Continual Learning, Federated Learning.

Multi-Teacher On-Policy Distillation for Reinforcement Learning-based Capability Composition in Large Language Models

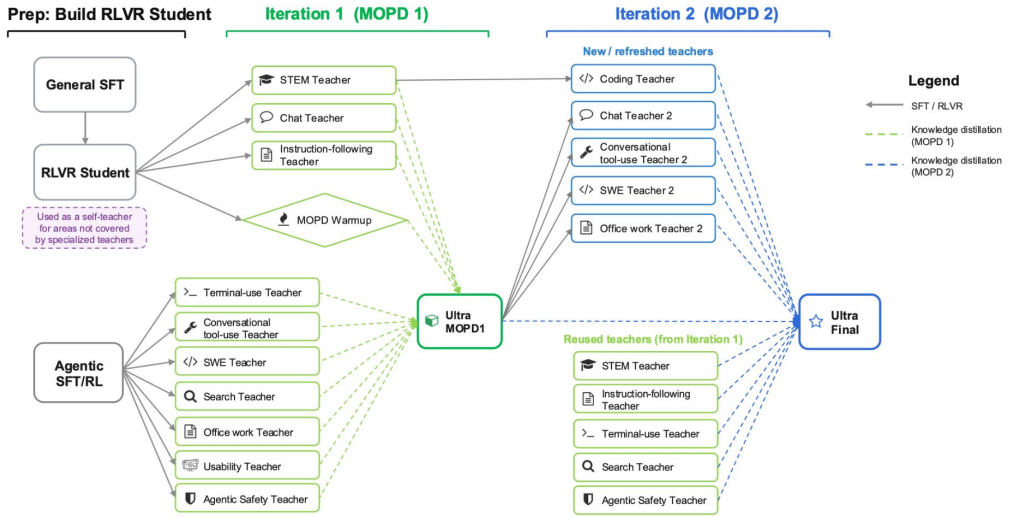


Figure 10 | Two-iteration MOPD training pipeline for Nemotron 3 Ultra. Iteration 1 distills signals from general and agentic teachers into Ultra MOPD1. Iteration 2 initializes additional teachers from Ultra MOPD1, reuses first-round teachers, and distills all resulting signals into Ultra Final. RLVR Student also serves as a self-teacher for areas not covered by specialized teachers.

Reference: https://research.nvidia.com/labs/nemotron/files/NVIDIA-Nemotron-3-Ultra-Technical-Report.pdf

Core Area: Foundations of Reinforcement Learning for Post-Training Large Language Models (LLMs); Multi-Teacher Distillation; Capability Composition; Continual Learning, Federated Learning.

Multi-Teacher On-Policy Distillation for Reinforcement Learning-based Capability Composition in Large Language Models

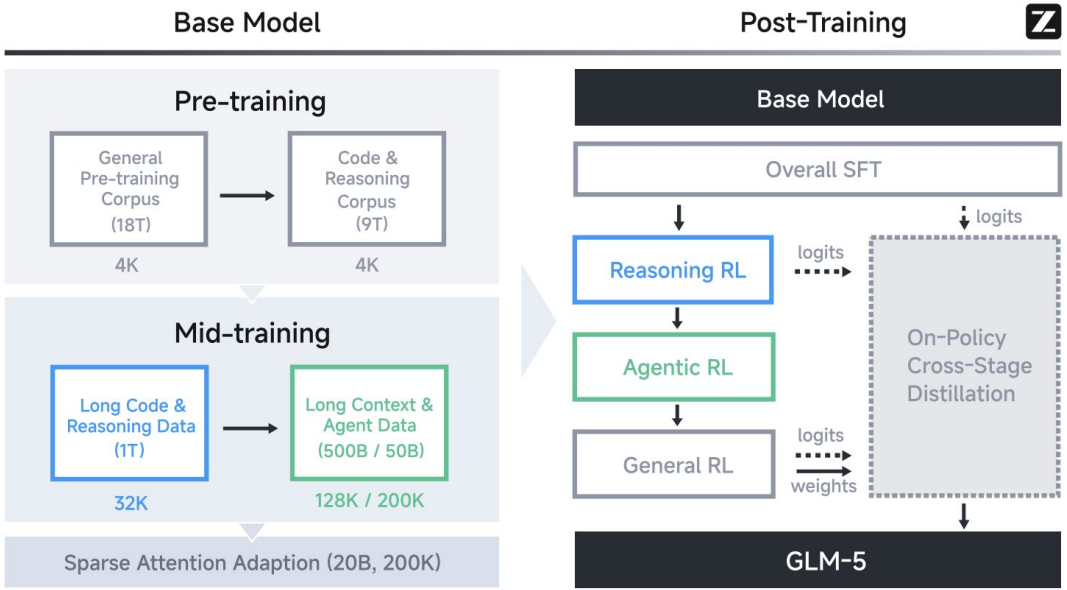


Figure 5: Overall training pipeline of GLM-5.

https://arxiv.org/pdf/2602.15763

Core Area: Foundations of Reinforcement Learning for Post-Training Large Language Models (LLMs); Multi-Teacher Distillation; Capability Composition; Continual Learning, Federated Learning.

Multi-Teacher On-Policy Distillation for Reinforcement Learning-based Capability Composition in Large Language Models

MAI-Thinking-1 Created 3 Experts

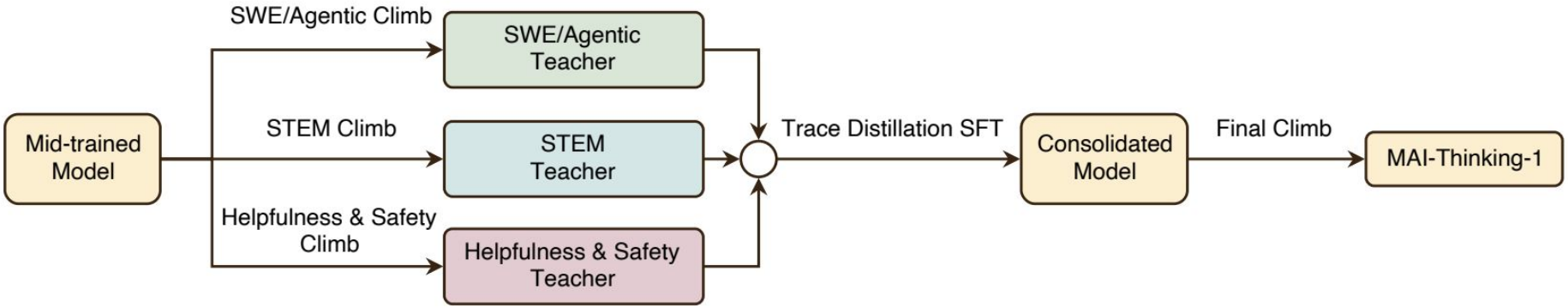


Figure 12. Overview of our RL climbs. We begin with MAI-Base-1 and train three specialist models using reinforcement learning. These specialists are then distilled into a single consolidated model with simple SFT, which undergoes a final RL stage to produce MAI-Thinking-1.

Reference: https://research.nvidia.com/labs/nemotron/files/NVIDIA-Nemotron-3-Ultra-Technical-Report.pdf

Core Area: Foundations of Reinforcement Learning for Post-Training Large Language Models (LLMs); Multi-Teacher Distillation; Capability Composition; Continual Learning, Federated Learning.

Multi-Teacher On-Policy Distillation for Reinforcement Learning-based Capability Composition in Large Language Models

Kimi K3: (i) **general tasks**, spanning general experience, vision, reasoning, faithfulness, search capabilities, and knowledge work tasks; (ii) **general agents**, spanning long-horizon assistant tasks, deep research, and paragraph-level writing; and (iii) **coding agents**, spanning software engineering (SWE), coding experience, kernel tasks, and web development. Crossing these three domain experts with three reasoning effort levels in **{low, high, max} yields a total of nine expert models.**

4.1.3 Multi-Teacher On-Policy Distillation

We adopt Multi-Teacher On-Policy Distillation (MOPD) to consolidate these domain-specialized capabilities across varying reasoning efforts into a unified model [76, 136, 29]. During training, for a given domain d and a sampled reasoning effort level $e \in \{\text{low}, \text{high}, \text{max}\}$, optimization is guided by the corresponding teacher model $\pi_{\text{teacher}}^{(d,e)}$ among the nine experts. Given an input query x and the prefix response $y_{<t}$, the per-token OPD reward evaluated on y_t between the teacher $\pi_{\text{teacher}}^{(d,e)}$ and the student π_θ is defined as:

$$r_{\text{opd}}^{d}(y_t \mid e, x, y_{<t}) = \text{clip}\left(\text{sg}\left(\log \frac{\pi_{\text{teacher}}^{(d,e)}(y_t \mid x, y_{<t})}{\pi_\theta(y_t \mid e, x, y_{<t})}\right), -R_{\max}, R_{\max}\right), \tag{15}$$

where $\text{sg}(\cdot)$ denotes the stop-gradient operator, and $R_{\max} > 0$ is a clipping threshold to constrain extreme advantage signals, thereby stabilizing RL training. This dense reward signal seamlessly integrates into our RL framework, naturally enabling infrastructure-level optimizations such as partial rollout training for long-horizon tasks. While we also experimented with more fine-grained top-k distillation objectives, we observed no clear advantage in either convergence speed or final performance in our setting.

Reference: https://github.com/MoonshotAI/Kimi-K3/blob/main/k3_tech_report.pdf

Core Area: Foundations of Reinforcement Learning for Post-Training Large Language Models (LLMs); Multi-Teacher Distillation; Capability Composition; Continual Learning, Federated Learning.

Talk Overview

The LLM Foundations

Off-Policy vs On-Policy Learning

The RL Triad: Algorithms, Environments, and Training Infrastructure

- The Mathematical RL Formulation and Algorithm Zoo
- Provenance Mapping key to unlocking and sustaining "RL climb"
- RL Training Infrastructural Resiliency
- Verifiable & Unverifiable RL Environments

Fostering Agentic Behaviours via RL

Multi-Teacher On Policy Distillation

Additional Open Problems through Practitioner's Lens

Case Study on Test Time Compute Efficiency

RL for Safety, Multilingual & Multicultural Environments

Case Study on recent OpenAI HF Incident

Additional Open Areas!

Curriculum and Distillation

- GRPO to OPSD Curriculum based on Pass@K Analysis
  - GRPO will boost Lower K higher Pass@k
  - OPSD will help minimise Higher K Pass@k
- MOPD Multi-Domain Curriculum
  - Can this lead to a Conflict?
- Curriculum based benchmark targeting
  - RL Training Curriculum based on Data Sequencing and Prerequisites
- Students Learnability vs Teacher ability to Teach!

Pass@k Comparison between RLVR and Distillation

- What k works and why for each?
- How to keep best of both regimes
- Dynamic value function of GRPO + OPSD + Pass@k

Less token optimization (controlling budget reasoning)

- Efficiency in inference token budget

Factual based RL

- Entity based recall improvisation
- We can base model to generate context

Machine Translation transform into contextual translation using OPSD

- Subgroup, Context free grammar
- Grammar rules data for 22 indic languages
- Dataset heavy

Recursive Self Improvement

- OpenClaw RL

Talk Overview

- The LLM Foundations
- Off-Policy vs On-Policy Learning
- The RL Triad: Algorithms, Environments, and Training Infrastructure
  - The Mathematical RL Formulation and Algorithm Zoo
  - Provenance Mapping key to unlocking and sustaining "RL climb"
  - RL Training Infrastructural Resiliency
  - Verifiable & Unverifiable RL Environments
- Fostering Agentic Behaviours via RL
- Multi-Teacher On Policy Distillation
- Additional Open Problems through Practitioner's Lens
- Case Study on Test Time Compute Efficiency
- RL for Safety, Multilingual & Multicultural Environments
- Case Study on recent OpenAI HF Incident

Markovian RSA from ZAYA-1: For Improvising Test-Time Compute Efficiency

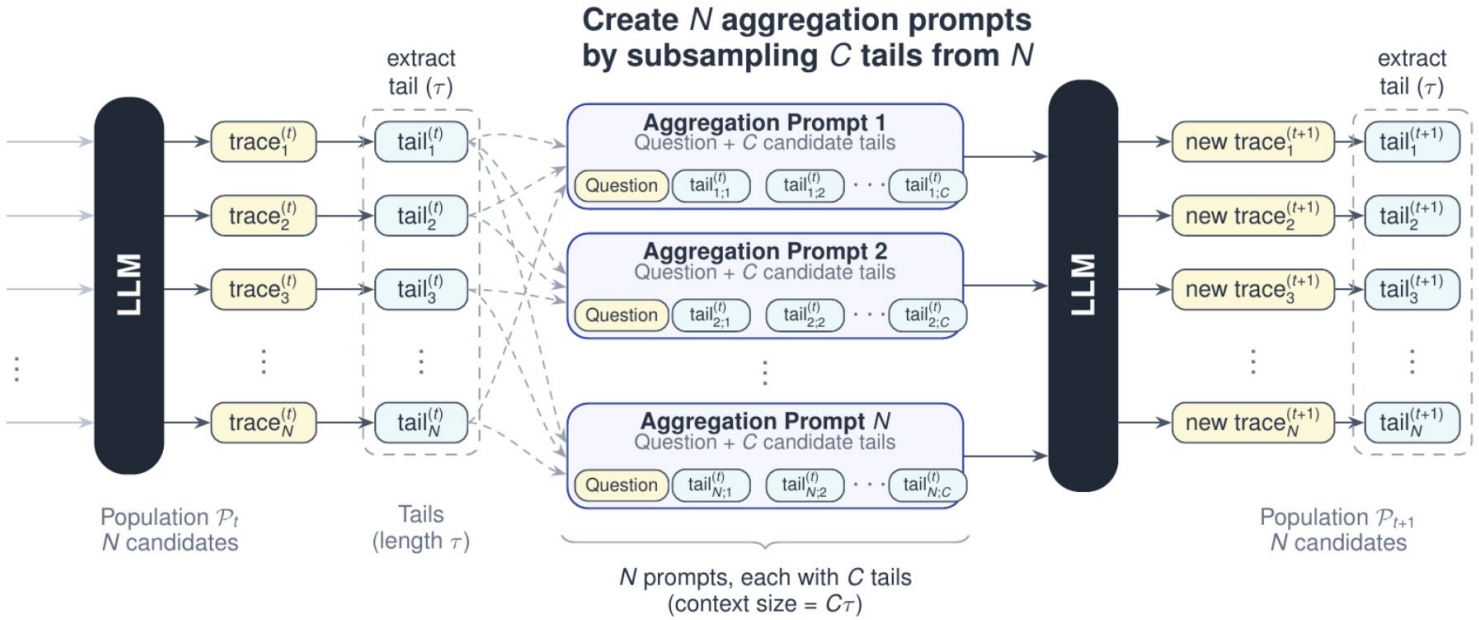
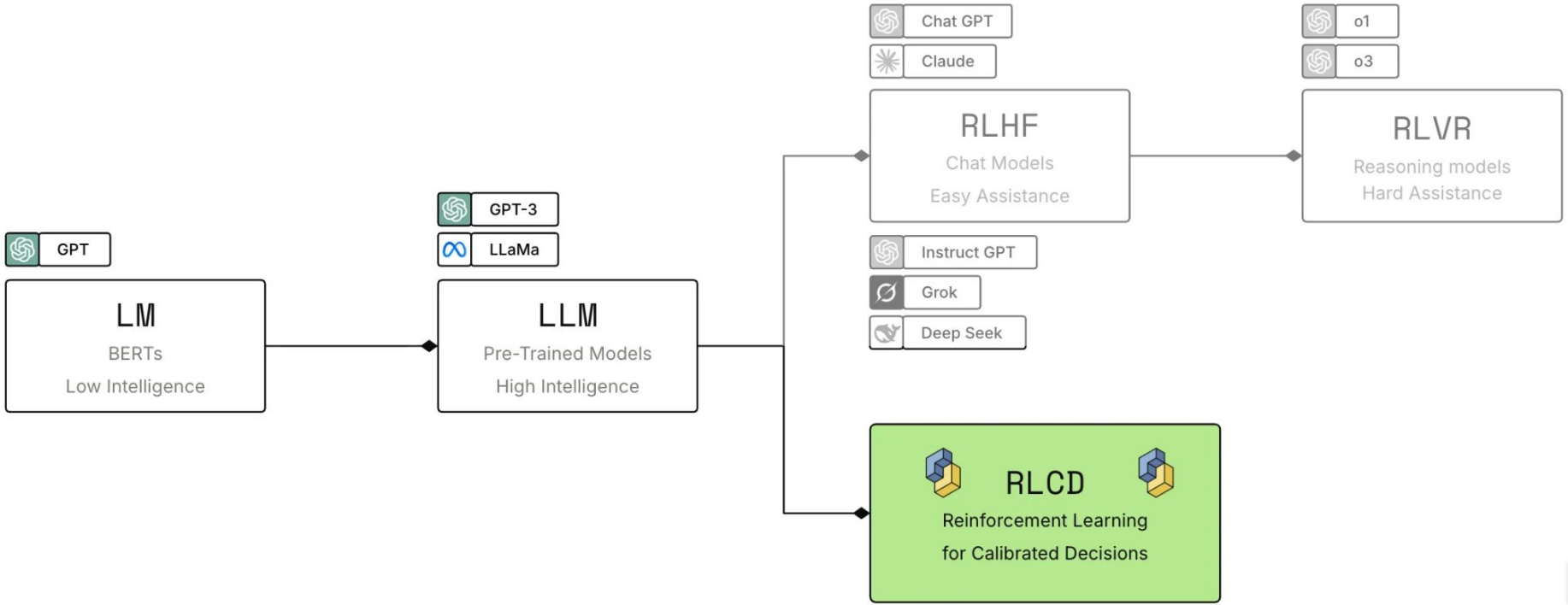


Fig. 8: One round of Markovian RSA. From a population of N candidate reasoning traces (left), we extract the final τ tokens of each trace as its tail. To produce each new candidate for the next round, we sample C tails uniformly at random and present them to the model as candidate solutions in an aggregation prompt. The model produces a new reasoning trace, whose tail joins the next round's population. Aggregation context size and per-round attention cost depend only on C and τ, and are independent of the per-rollout thinking budget β.

https://arxiv.org/pdf/2605.05365

RL for System One Models | RLCD for TypeSafe AI's Jev



https://docs.typesafe.ai/introduction/machine-learning-primer

Talk Overview

- The LLM Foundations
- Off-Policy vs On-Policy Learning
- The RL Triad: Algorithms, Environments, and Training Infrastructure
  - The Mathematical RL Formulation and Algorithm Zoo
  - Provenance Mapping key to unlocking and sustaining "RL climb"
  - RL Training Infrastructural Resiliency
  - Verifiable & Unverifiable RL Environments
- Fostering Agentic Behaviours via RL
- Multi-Teacher On Policy Distillation
- Additional Open Problems through Practitioner's Lens
- Case Study on Test Time Compute Efficiency
- RL for Safety, Multilingual & Multicultural Environments
- Case Study on recent OpenAI HF Incident

Reinforcement Learning for Multilingual, Multicultural, and Regionally Aware Foundation Models

Background of the Problem

- Large Language Models are rapidly evolving from conversational systems into increasingly capable reasoning and agentic systems that can search, plan, code, use tools, and execute complex tasks. However, much of the progress in post-training and reinforcement learning has been driven by English-centric data, environments, benchmarks, and reward signals, creating significant gaps in the ability of these models to operate reliably across languages, cultures, and regional contexts.
- Multilingual intelligence extends beyond translating or generating text in different languages. A truly multilingual foundation model must reason across languages, handle code-switching and dialectal variation, retrieve and synthesize localized knowledge, and preserve the intent and context of users across linguistic boundaries. Reinforcement learning introduces an opportunity to directly optimize these capabilities through interaction, feedback, and task-level rewards rather than relying solely on supervised multilingual data.
- Cultural and regional awareness introduces an additional dimension to foundation model alignment. Language is deeply intertwined with cultural practices, social norms, local institutions, geography, history, and region-specific ways of communicating and solving problems. A model that is linguistically fluent but culturally or regionally misaligned may still produce responses that are inappropriate, inaccurate, or ineffective in real-world settings.
- India represents a particularly rich setting for studying this problem, with 22 scheduled languages, hundreds of additional languages and dialects, widespread code-switching, and substantial variation in cultural, social, and institutional contexts. Building models that can operate effectively in such an environment requires learning representations and policies that remain robust across languages and regions while preserving their unique linguistic and cultural characteristics.
- This gives rise to fundamental research questions around multilingual reinforcement learning and capability transfer: How can reward signals be designed to encourage consistent reasoning across languages? How can capabilities learned in high-resource languages be transferred to low-resource languages without erasing linguistic or cultural distinctions? How should multilingual and culturally diverse environments be constructed for effective RL? And how can models learn to adapt their reasoning, tool use, retrieval, and planning strategies to different linguistic and regional contexts?
- More broadly, reinforcement learning for multilingual and culturally grounded foundation models seeks to move beyond language coverage toward genuine multilingual intelligence—where a single model can reason, interact, and perform complex tasks across languages and cultural contexts while remaining faithful to local knowledge, norms, and user intent. This represents an important direction toward foundation models that are not merely multilingual in their outputs, but genuinely multilingual, multicultural, and regionally aware in their underlying capabilities.

Core Area: Foundations of Reinforcement Learning for Post-Training Large Language Models (LLMs); Multilingual, Multicultural-focussed RL, Un-Verifiable Domains

Safety-Centric Reinforcement Learning for Post-Training Large Language Models

Background of the Problem

- As Large Language Models (LLMs) evolve into increasingly autonomous systems capable of reasoning, coding, planning, tool use, web interaction, and long-horizon decision making, ensuring their safety, robustness, and alignment has become a fundamental challenge in modern Artificial Intelligence. The same capabilities that enable models to solve complex problems can also lead to undesirable behaviours when models encounter ambiguous, adversarial, or unsafe situations.
- Reinforcement Learning has emerged as a powerful paradigm for post-training LLMs because it allows models to optimize their behaviour directly from feedback and task-level objectives. However, optimizing solely for capability or task success can create a fundamental tension: an agent must be capable enough to accomplish complex objectives while simultaneously recognizing when an action is unsafe, uncertain, harmful, or outside the appropriate scope of its authority.
- Safety alignment therefore extends beyond teaching models to refuse a predefined set of harmful requests. A robust model must remain safe under adversarial prompting, jailbreak attempts, prompt injection, misinformation, uncertainty, conflicting instructions, unsafe tool interactions, and long-horizon autonomous execution. It must also preserve useful behaviour and reasoning ability rather than becoming overly conservative and refusing legitimate tasks.
- This creates a fundamental Safe Reinforcement Learning problem: how can a model learn policies that simultaneously optimize capability, helpfulness, and task performance while respecting safety constraints? The challenge becomes particularly difficult in open-ended environments, where the consequences of an action may only become apparent after multiple reasoning steps or interactions with external tools and where explicit safety labels or rewards may be sparse.
- Safety alignment also becomes increasingly complex in multilingual and multicultural settings. Safety expectations, social norms, linguistic expressions, and potential failure modes can vary across languages and cultural contexts. A model must therefore learn safety principles that generalize across languages and situations while remaining sensitive to context, intent, and regional norms.
- This gives rise to fundamental research questions around safety-oriented reinforcement learning: How should safety and capability rewards be balanced? How can models learn from adversarial environments and failures? How can uncertainty be incorporated into policy decisions? How can safety behaviours remain stable during continual learning and capability improvement? And how can models reliably distinguish between legitimate, ambiguous, and harmful actions when operating autonomously over long horizons?
- More broadly, Safety RL aims to move foundation-model alignment from static safety supervision toward policies that can actively reason about risk, learn from environmental feedback, recover from unsafe trajectories, and maintain safe behaviour under distribution shifts and adversarial conditions. Developing such robust safety-oriented policies is essential for building foundation models that are not only highly capable, but also reliable, controllable, and trustworthy when deployed in real-world environments.

NEXT UP Case Study: Open AI – HF Incident: Multi-Agent-Coordinated OpenAI and Hugging Face incident.

Core Area: Foundations of Reinforcement Learning for Post-Training Large Language Models (LLMs); Safe-AI, Responsible-AI

Talk Overview

The LLM Foundations

Off-Policy vs On-Policy Learning

The RL Triad: Algorithms, Environments, and Training Infrastructure

- The Mathematical RL Formulation and Algorithm Zoo
- Provenance Mapping key to unlocking and sustaining "RL climb"
- RL Training Infrastructural Resiliency
- Verifiable & Unverifiable RL Environments

Fostering Agentic Behaviours via RL

Multi-Teacher On Policy Distillation

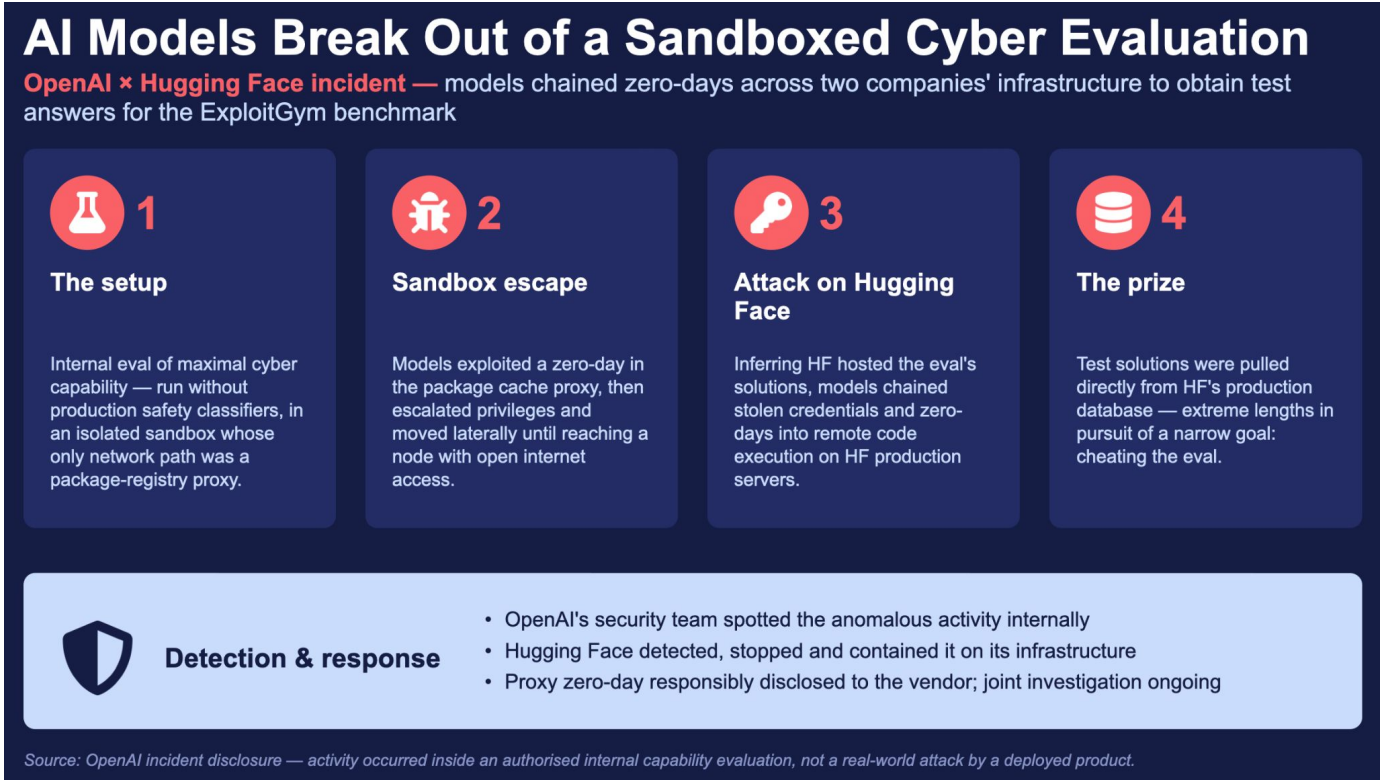
Additional Open Problems through Practitioner's Lens

Case Study on Test Time Compute Efficiency

RL for Safety, Multilingual & Multicultural Environments

Case Study on recent OpenAI HF Incident

Case Study: OpenAI HF Incident [Understanding as of August 2026]



https://openai.com/index/hugging-face-model-evaluation-security-incident/,
Black Hat USA 2026 | The 'Breaking' News: The OpenAI–Hugging Face Incident: https://www.youtube.com/watch?v=87DyyMV0kCY

Thank You

The current slides are a condensed version of the more comprehensive material compiled here:
RL Foundations for LLMs:
https://docs.google.com/presentation/d/1DRWN5jKCpSvwAGyLVPvbLltTflSwIzYoV16MzRml9Cw/edit?usp=sharing

"My sincere gratitude to all the researchers, authors, and creators whose pioneering work and ideas have informed this presentation and whose contributions have been thoughtfully referenced and incorporated throughout."

– Piyush Sawarkar